https://wikidocs.net/7014

print 함수의 sep 인자로 ";"를 입력하면 출력되는 값들 사이에 한 칸의 공백대신 세미콜론이 출력됩니다. print("naver", "kakao", "samsung", sep=";")

print() 함수를 사용하여 다음과 같이 출력하세요.
naver/kakao/sk/samsung

정답확인
print("naver", "kakao", "samsung", sep="/")

다음 코드를 수정하여 줄바꿈이 없이 출력하세요. (힌트: end='') print 함수는 두 번 사용합니다. 세미콜론 (;)은 한줄에 여러 개의 명령을 작성하기 위해 사용합니다.

print("first");print("second")
정답확인
print("first", end=""); print("second")

변수 s와 t에는 각각 문자열이 바인딩 되어있습니다.
>> s = "hello"
>> t = "python"
두 변수를 이용하여 아래와 같이 출력해보세요.

실행 예:
hello! python
정답확인
s = "hello"
t = "python"
print(s+"!", t)
print(s+"! "+t)
---
015 type 함수
type() 함수는 데이터 타입을 판별합니다. 변수 a에는 128 숫자가 바인딩돼 있어 type 함수가 int (정수)형임을 알려줍니다.

>> a = 128
>> print (type(a))
<class 'int'>
아래 변수에 바인딩된 값의 타입을 판별해보세요.

>> a = "132"

정답확인
a = "132"
print(type(a))
---

016 문자열을 정수로 변환
문자열 '720'를 정수형으로 변환해보세요.

>> num_str = "720"
정답확인
num_str = "720" #형변환
num_int = int(num_str)
print(num_int+1, type(num_int))

721 <class 'int'>
---
017 정수를 문자열 100으로 변환
정수 100을 문자열 '100'으로 변환해보세요.

정답확인
num = 100
result = str(num)
print(result, type(result))

100 <class 'str'>
---
018 문자열을 실수로 변환
문자열 "15.79"를 실수(float) 타입으로 변환해보세요.

정답확인
data = "15.79"
data = float(data)
print(data, type(data))

15.79 <class 'float'>
---
019 문자열을 정수로 변환
year라는 변수가 문자열 타입의 연도를 바인딩하고 있습니다. 이를 정수로 변환한 후 최근 3년의 연도를 화면에 출력해보세요.

year = "2020"

정답확인
year = "2020"
print(int(year)-3) # 2017
print(int(year)-2) # 2018
print(int(year)-1) # 2019

2017
2018
2019
---
021 문자열 인덱싱
letters가 바인딩하는 문자열에서 첫번째와 세번째 문자를 출력하세요.

>> letters = 'python'
실행 예
p t
정답확인
파이썬 문자열에서 한 글자를 가져오는 것을 인덱싱이라고 부릅니다. 파이썬 인덱싱은 0부터 시작합니다.

lang = 'python'
print(lang[0], lang[2])
---
022 문자열 슬라이싱
자동차 번호가 다음과 같을 때 뒤에 4자리만 출력하세요.

>> license_plate = "24가 2210"
실행 예: 2210
정답확인
문자열에서 여러 글자를 가져오는 것을 슬라이싱이라고 부릅니다. 음수 값은 문자열의 뒤에서부터 인덱싱 또는 슬라이싱함을 의미합니다. 슬라이싱에서 시작 인덱스를 생략혀면 0으로 간주하고 끝 인덱스를 생략하면 문자열의 끝을 의미합니다.

license_plate = "24가 2210"
print(license_plate[-4:])
---
023 문자열 인덱싱
아래의 문자열에서 '홀' 만 출력하세요.

>> string = "홀짝홀짝홀짝"
실행 예:

홀홀홀
정답확인
슬라이싱할 때 시작인덱스:끝인덱스:오프셋을 지정할 수 있습니다.

```
string = "홀짝홀짝홀짝"
print(string[::2])
```

---

024 문자열 슬라이싱
문자열을 거꾸로 뒤집어 출력하세요.

```
>> string = "PYTHON"
```

실행 예:
NOHTYP
정답확인

```
string = "PYTHON"
print(string[::-1])
```

---

025 문자열 치환
아래의 전화번호에서 하이폰('-')을 제거하고 출력하세요.

```
>> phone_number = "010-1111-2222"
```

실행 예
010 1111 2222
정답확인
파이썬 문자열에서 replace 메서드를 사용하면 문자열을 일부를 치환할 수 있습니다. 문자열은 수정할 수 없는 자료형이므로 기존 문자열은 그대로 두고 치환된 새로운 문자열이 리턴됩니다.

```
phone_number = "010-1111-2222"
phone_number1 = phone_number.replace("-", " ")
print(phone_number1)
```

---

026 문자열 다루기
25번 문제의 전화번호를 아래와 같이 모두 붙여 출력하세요.

실행 예
01011112222
정답확인

```
phone_number = "010-1111-2222"
phone_number1 = phone_number.replace('-', '')
print(phone_number1)
```

---
027 문자열 다루기
url 에 저장된 웹 페이지 주소에서 도메인을 출력하세요.

```
>> url = "http://sharebook.kr"
```

실행 예:
kr
정답확인
문자열로 표현된 url에서 .을 기준으로 분리합니다. 분리된 url 중 마지막을 인덱싱하면 도메인만 출력할 수 있습니다.

```
url = "http://sharebook.kr"
url_split = url.split('.')
print(url_split[-1])
```

---
028 문자열은 immutable (불변의)
아래 코드의 실행 결과를 예상해보세요.

```
>> lang = 'python'
>> lang[0] = 'P'
>> print(lang)
```

정답확인
문자열은 수정할 수 없습니다. 실행 결과를 확인해보면 문자열이 할당(assignment) 메서드를 지원하지 않음을 알 수 있습니다.(직접 입력하여 고칠 수는 없고 replace로 새변수로 가능)

```
TypeError          Traceback (most recent call last)
<ipython-input-22-a0f3d05b669c> in <module>
      1 lang = "python"
----> 2 lang[0]= "P"
      3 print(lang)

TypeError: 'str' object does not support item assignment
```

==========
029 replace 메서드
아래 문자열에서 소문자 'a'를 대문자 'A'로 변경하세요.

```
>> string = 'abcdfe2a354a32a'
```

실행 예:
Abcdfe2A354A32A
정답확인
string = 'abcdfe2a354a32a'

```
string = string.replace('a', 'A')
print(string)
```

---

030 replace 메서드

아래 코드의 실행 결과를 예상해보세요.

```
>> string = 'abcd'
>> string.replace('b', 'B')
>> print(string)
```

정답확인

abcd가 그대로 출력됩니다. 왜냐하면 문자열은 변경할 수 없는 자료형이기 때문입니다. replace 메서드를 사용하면 원본은 그대로 둔채로 변경된 새로운 문자열 객체를 리턴해줍니다.

abcd

---

031 문자열 합치기

아래 코드의 실행 결과를 예상해보세요.

```
>> a = "3"
>> b = "4"
>> print(a + b)
```

정답확인

두 문자열에 대해 덧셈 기호는 문자열의 연결을 의미합니다. 따라서 "34"라는 새로운 문자열이 생성되고 그 값이 print 함수에 의해 화면에 출력됩니다.

---

032 문자열 곱하기

아래 코드의 실행 결과를 예상해보세요.

>> print("Hi" * 3)
정답확인
문자열에 대한 곱셈은 문자열의 반복을 의미합니다. 따라서 다음과 같이 문자열이 출력됩니다.

HiHiHi
---
033 문자열 곱하기
화면에 '-'를 80개 출력하세요.

실행 예:

정답확인
print("-" * 80)
---
034 문자열 곱하기
변수에 다음과 같은 문자열이 바인딩되어 있습니다.

>>> t1 = 'python'
>>> t2 = 'java'
변수에 문자열 더하기와 문자열 곱하기를 사용해서 아래와 같이 출력해보세요.

실행 예:
python java python java python java python java
정답확인
t1 = "python"
t2 = "java"
t3 = t1 + ' ' + t2 + ' '
print(t3 * 4)

---
035 문자열 출력
변수에 다음과 같이 문자열과 정수가 바인딩되어 있을 때 % formatting을 사용해서 다음과 같이 출력해보세요.

name1 = "김민수"
age1 = 10
name2 = "이철희"
age2 = 13
이름: 김민수 나이: 10

이름: 이철희 나이: 13
정답확인
print 포맷팅에서 %s는 문자열 데이터 타입의 값을 %d는 정수형 데이터 타입 값의 출력을 의미합니다.

```
name1 = "김민수"
age1 = 10
name2 = "이철희"
age2 = 13
print("이름: %s 나이: %d" % (name1, age1))
print("이름: %s 나이: %d" % (name2, age2))
```

---

036 문자열 출력
문자열의 format( ) 메서드를 사용해서 035번 문제를 다시 풀어보세요.

정답확인
문자열의 포맷 메서드는 타입과 상관없이 값이 출력될 위치에 { }를 적어주면 됩니다.

```
name1 = "김민수"
age1 = 10
name2 = "이철희"
age2 = 13
print("이름: {} 나이: {}".format(name1, age1))
print("이름: {} 나이: {}".format(name2, age2))
```

---
037 문자열 출력
파이썬 3.6부터 지원하는 f-string을 사용해서 035번 문제를 다시 풀어보세요.

정답확인
f-string은 문자열 앞에 f가 붙은 형태입니다. f-string을 사용하면 {변수}와 같은 형태로 문자열 사이에 타입과 상관없이 값을 출력할 수 있습니다.

```
name1 = "김민수"
age1 = 10
name2 = "이철희"
age2 = 13
print(f"이름: {name1} 나이: {age1}")
print(f"이름: {name2} 나이: {age2}")
```

---
038 컴마 제거하기

삼성전자 상장주식수가 다음과 같습니다. 컴마 제거한 후 이를 정수 타입으로 변환해보세요.
상장주식수 = "5,969,782,550"
정답확인
정수형으로 타입을 변환하려면 int( ) 함수를 사용하면 됩니다. 이때 숫자 형태의 문자열에 컴마가 있는 경우 바로 변환되지 않습니다. 먼저 문자열의 replace 메서드로 컴마를 제거한 후 변환해야합니다.

```
상장주식수 = "5,969,782,550"
컴마제거 = 상장주식수.replace(",", "")
타입변환 = int(컴마제거)
print(타입변환, type(타입변환))
```

```
5969782550 <class 'int'>
```

---

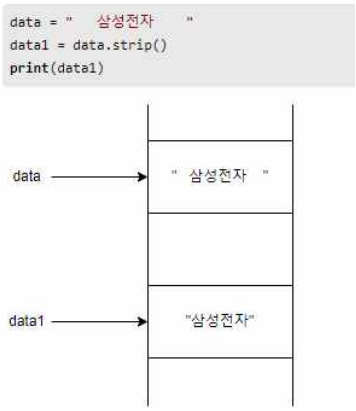
039 문자열 슬라이싱
다음과 같은 문자열에서 '2020/03'만 출력하세요.
분기 = "2020/03(E) (IFRS연결)"
정답확인
문자열에서 슬라이싱을 사용하면 여러 글자를 접근할 수 있습니다.

```
분기 = "2020/03(E) (IFRS연결)"
print(분기[:7])
```

---

040 strip 메서드
문자열의 좌우의 공백이 있을 때 이를 제거해보세요.data = " 삼성전자 "
정답확인
문자열에서 strip( ) 메서드를 사용하면 좌우의 공백을 제거할 수 있습니다. 이때 원본 문자열은 그대로 유지되고 공백이 제거된 새로운 문자열이 반환됩니다.

```
data = "  삼성전자  "
data1 = data.strip()
print(data1)
```



---
041 upper 메서드
다음과 같은 문자열이 있을 때 이를 대문자 BTC_KRW로 변경하세요.

ticker = "btc_krw"
정답확인
upper 메서드를 호출하면 문자열을 대문자로 만들 수 있습니다. 다만 이 경우에도 원본 문자열은 유지되고 대문자로 변경된 새로운 문자열 객체가 반환되는 겁니다. 반환된 새로운 객체를 새로운 변수로 바인딩한 후 이를 print 함수로 출력하면 됩니다.

```
ticker = "btc_krw"
ticker1 = ticker.upper()
print(ticker1)
```
---
042 lower 메서드
다음과 같은 문자열이 있을 때 이를 소문자 btc_krw로 변경하세요.

ticker = "BTC_KRW"
정답확인
lower 메서드를

```
ticker = "BTC_KRW"
ticker2 = ticker.lower()
print(ticker2)
```
---
043 capitalize 메서드
문자열 'hello'가 있을 때 이를 'Hello'로 변경해보세요.

정답확인
```
a = "hello"
a = a.capitalize()
print(a)
```
---
044 endswith 메서드
파일 이름이 문자열로 저장되어 있을 때 endswith 메서드를 사용해서 파일 이름이 'xlsx'로 끝나는지 확인해보세요.

file_name = "보고서.xlsx"
정답확인
```
file_name = "보고서.xlsx"
file_name.endswith("xlsx")
```

---

046 **startswith 메서드**

파일 이름이 문자열로 저장되어 있을 때 startswith 메서드를 사용해서 파일 이름이 '2020'로 시작하는지 확인해보세요.

file_name = "2020_보고서.xlsx"

정답확인

file_name = "2020_보고서.xlsx"

file_name.startswith("2020")

---

047 **split** 메서드

다음과 같은 문자열이 있을 때 공백을 기준으로 문자열을 나눠보세요.

a = "hello world"

정답확인

문자열의 split() 메서드를 사용하면 문자열에서 공백을 기준으로 분리해줍니다.

a = "hello world"

a.split()

---

048 split 메서드

다음과 같이 문자열이 있을 때 btc와 krw로 나눠보세요.

ticker = "btc_krw"

정답확인

문자열에서 split() 메서드는 문자열을 분리할 때 사용합니다. 이때 어떤 값을 넘겨주면 그 값을 기준으로 문자열을 분리해줍니다.

ticker = "btc_krw"

ticker.split("_")

---

049 split 메서드

다음과 같이 날짜를 표현하는 문자열이 있을 때 연도, 월, 일로 나눠보세요.

date = "2020-05-01"

정답확인

date = "2020-05-01"

date.split("-")

---

050 **rstrip** 메서드

문자열의 오른쪽에 공백이 있을 때 이를 제거해보세요.

data = "039490 "
정답확인
rstrip() 메서드를 사용하면 오른쪽 공백이 제거된 새로운 문자열 객체가 반환됩니다. 그 값을 **data라는 변수가 새로 바인딩합니다.** 기존의 공백이 포함된 문자열은 메모리에서 자동으로 삭제됩니다.

```
data = "039490  "
data = data.rstrip()
```

---
051 리스트 생성
2016년 11월 영화 예매 순위 기준 top3는 다음과 같습니다. 영화 제목을 movie_rank 이름의 리스트에 저장해보세요. (순위 정보는 저장하지 않습니다.)

순위	영화
1	닥터 스트레인지
2	스플릿
3	럭키

정답확인
영화 제목은 문자열로 표현 가능합니다. 여러 개의 값을 저장하기 위해 파이썬 리스트 자료형을 사용합니다.

movie_rank = ["닥터 스트레인지", "스플릿", "럭키"]
---
052 리스트에 원소 추가
051의 movie_rank 리스트에 "배트맨"을 추가하라.

정답확인
movie_rank = ["닥터 스트레인지", "스플릿", "럭키"]
movie_rank.append("배트맨")
print(movie_rank)

053
movie_rank 리스트에는 아래와 같이 네 개의 영화 제목이 바인딩되어 있다. "슈퍼맨"을 "닥터 스트레인지"와 "스플릿" 사이에 추가하라.

movie_rank = ['닥터 스트레인지', '스플릿', '럭키', '배트맨']
정답확인
리스트의 insert(인덱스, 원소) 메서드를 사용하면 특정 위치에 값을 끼어넣기 할 수 있습니다.

movie_rank = ['닥터 스트레인지', '스플릿', '럭키', '배트맨']
movie_rank.insert(1, "슈퍼맨")
print(movie_rank)

054
movie_rank 리스트에서 '럭키'를 삭제하라.

movie_rank = ['닥터 스트레인지', '슈퍼맨', '스플릿', '럭키', '배트맨']
정답확인
movie_rank = ['닥터 스트레인지', '슈퍼맨', '스플릿', '럭키', '배트맨']
del movie_rank[3]
print(movie_rank)

055
movie_rank 리스트에서 '스플릿' 과 '배트맨'을 를 삭제하라.

movie_rank = ['닥터 스트레인지', '슈퍼맨', '스플릿', '배트맨']
정답확인
del을 이용하여 리스트에서 원소를 삭제할 수 있습니다. 리스트에서 어떤 값을 삭제하면 남은 값들은 새로 인덱싱됩니다. 따라서 여러 값을 삭제할 때는 어떤 값이 먼저 삭제된 후 남은 원소들에 대해서 순서를 새로 고려한 후 삭제해야 합니다.

movie_rank = ['닥터 스트레인지', '슈퍼맨', '스플릿', '배트맨']
del movie_rank[2]
del movie_rank[2]
print(movie_rank)

056
lang1과 lang2 리스트가 있을 때 lang1과 lang2의 원소를 모두 갖고 있는 langs 리스트를 만들어라.

```
>> lang1 = ["C", "C++", "JAVA"]
>> lang2 = ["Python", "Go", "C#"]
```

실행 예:

```
>> langs
['C', 'C++', 'JAVA', 'Python', 'Go', 'C#']
```

정답확인

두 리스트를 더하면 새로운 리스트가 생성됩니다.

```
lang1 = ["C", "C++", "JAVA"]
lang2 = ["Python", "Go", "C#"]
langs = lang1 + lang2
print(langs)
```

057
다음 리스트에서 최댓값과 최솟값을 출력하라. (힌트: min(), max() 함수 사용)

```
nums = [1, 2, 3, 4, 5, 6, 7]
```

실행 예:

```
max:  7
min:  1
```

정답확인

```
nums = [1, 2, 3, 4, 5, 6, 7]
print("max: ", max(nums))
print("min: ", min(nums))
```

058
다음 리스트의 합을 출력하라.

```
nums = [1, 2, 3, 4, 5]
```

실행 예:

```
15
```

정답확인

```
nums = [1, 2, 3, 4, 5]
print(sum(nums))
```

059

다음 리스트에 저장된 데이터의 개수를 화면에 구하라.

cook = ["피자", "김밥", "만두", "양념치킨", "족발", "피자", "김치만두", "쫄면", "쏘세지", "라면", "팥빙수", "김치전"]
정답확인
cook = ["피자", "김밥", "만두", "양념치킨", "족발", "피자", "김치만두", "쫄면", "쏘세지", "라면", "팥빙수", "김치전"]
print(len(cook))

060
다음 리스트의 평균을 출력하라.

nums = [1, 2, 3, 4, 5]
실행 예:
3.0
정답확인
nums = [1, 2, 3, 4, 5]
average = sum(nums) / len(nums)
print(average)

061
price 변수에는 날짜와 종가 정보가 저장돼 있다. 날짜 정보를 제외하고 가격 정보만을 출력하라. (힌트 : 슬라이싱)

price = ['20180728', 100, 130, 140, 150, 160, 170]
출력 예시:
[100, 130, 140, 150, 160, 170]
정답확인
price = ['20180728', 100, 130, 140, 150, 160, 170]
print(price[1:])

062
슬라이싱을 사용해서 홀수만 출력하라.

nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
실행 예:
[1, 3, 5, 7, 9]
정답확인
nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(nums[::2])

063
슬라이싱을 사용해서 짝수만 출력하라.

nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
실행 예:
[2, 4, 6, 8, 10]
정답확인
nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print(nums[1::2])

064
슬라이싱을 사용해서 리스트의 숫자를 역 방향으로 출력하라.

nums = [1, 2, 3, 4, 5]
실행 예:
[5, 4, 3, 2, 1]
정답확인
nums = [1, 2, 3, 4, 5]
print(nums[::-1])

065
interest 리스트에는 아래의 데이터가 바인딩되어 있다.

interest = ['삼성전자', 'LG전자', 'Naver']
interest 리스트를 사용하여 아래와 같이 화면에 출력하라.

출력 예시:
삼성전자 Naver
정답확인
interest = ['삼성전자', 'LG전자', 'Naver']
print(interest[0], interest[2])
---
066 join 메서드
interest 리스트에는 아래의 데이터가 바인딩 되어 있다.

interest = ['삼성전자', 'LG전자', 'Naver', 'SK하이닉스', '미래에셋대우']
interest 리스트를 사용하여 아래와 같이 화면에 출력하라.

출력 예시:
삼성전자 LG전자 Naver SK하이닉스 미래에셋대우
정답확인

interest = ['삼성전자', 'LG전자', 'Naver', 'SK하이닉스', '미래에셋대우']
print(" ".join(interest))
---
067 join 메서드
interest 리스트에는 아래의 데이터가 바인딩 되어 있다.

interest = ['삼성전자', 'LG전자', 'Naver', 'SK하이닉스', '미래에셋대우']
interest 리스트를 사용하여 아래와 같이 화면에 출력하라.

출력 예시:
삼성전자/LG전자/Naver/SK하이닉스/미래에셋대우
정답확인
문자열의 join 메서드를 사용하면 리스트를 문자열로 붙일 수 있습니다.

interest = ['삼성전자', 'LG전자', 'Naver', 'SK하이닉스', '미래에셋대우']
print("/".join(interest))
---
068 join 메서드
interest 리스트에는 아래의 데이터가 바인딩 되어 있다.

interest = ['삼성전자', 'LG전자', 'Naver', 'SK하이닉스', '미래에셋대우']
join() 메서드를 사용해서 interest 리스트를 아래와 같이 화면에 출력하라.

출력 예시:
삼성전자
LG전자
Naver
SK하이닉스
미래에셋대우
정답확인
interest = ['삼성전자', 'LG전자', 'Naver', 'SK하이닉스', '미래에셋대우']
print("\n".join(interest))
---
069 문자열 split 메서드
회사 이름이 슬래시 ('/')로 구분되어 하나의 문자열로 저장되어 있다.

string = "삼성전자/LG전자/Naver"
이를 interest 이름의 리스트로 분리 저장하라.

실행 예시
>> print(interest)

['삼성전자', 'LG전자', 'Naver']
정답확인
string = "삼성전자/LG전자/Naver"
interest = string.split("/")
print(interest)
---
070 리스트 정렬
리스트에 있는 값을 오름차순으로 정렬하세요.

data = [2, 4, 3, 1, 5, 10, 9]
정답확인
data = [2, 4, 3, 1, 5, 10, 9]
data.sort()
print(data)
data = [2, 4, 3, 1, 5, 10, 9]
data2 = sorted(data)
print(data2)

[1, 2, 3, 4, 5, 9, 10]
[1, 2, 3, 4, 5, 9, 10]

071
my_variable 이름의 비어있는 튜플을 만들라.

정답확인
괄호는 튜플을 정의하는 기호입니다.
my_variable = ()
정말 튜플이 생성됐는지 확인해 봅시다. type() 함수는 변수에 바인딩된 데이터의 타입을 반환합니다.

>> print(type(my_variable))
<class 'tuple'>

072
2016년 11월 영화 예매 순위 기준 top3는 다음과 같다. 영화 제목을 movie_rank 이름의 튜플에 저장하라. (순위 정보는 저장하지 않는다.)

순위	영화
1	닥터 스트레인지
2	스플릿
3	럭키

정답확인
movie_rank = ("닥터 스트레인지", "스플릿", "럭키")

073
숫자 1 이 저장된 튜플을 생성하라.

정답확인
아래와 같이 괄호와 함께 하나의 정숫값을 저장하면 튜플이 정의 될 것같지만 그렇지 않습니다. type()을 출력해보면 파이썬은 튜플이 아닌 정수로 인식합니다.
```
>> my_tuple = (1)
>> type (my_tuple)
int
```
하나의 데이터가 저장되는 경우, 아래와 같이 쉼표를 입력해만 합니다.

```
my_tuple = (1, )
```

074
다음 코드를 실행해보고 오류가 발생하는 원인을 설명하라.

```
>> t = (1, 2, 3)
>> t[0] = 'a'
Traceback (most recent call last):
  File "<pyshell#46>", line 1, in <module>
    t[0] = 'a'
TypeError: 'tuple' object does not support item assignment
```
정답확인
tuple은 원소(element)의 값을 변경할 수 없습니다.

075
아래와 같이 t에는 1, 2, 3, 4 데이터가 바인딩되어 있다. t가 바인딩하는 데이터 타입은 무엇인가?

```
t = 1, 2, 3, 4
```
정답확인
원칙적으로 튜플은 괄호와 함께 데이터를 정의해야 하지만, 사용자 편의를 위해 괄호 없이도 동작합니다.

076
변수 t에는 아래와 같은 값이 저장되어 있다. 변수 t가 ('A', 'b', 'c') 튜플을 가리키도록 수정하라.

t = ('a', 'b', 'c')
정답확인
튜플의 값은 변경할 수 없기 때문에, 리스트와 달리 아래 코드는 동작하지 않습니다.
t[0] = 'A'
새로운 튜플을 만들고 t 라는 변수를 업데이트 해야 합니다. 기존의 튜플 ('a', 'b', 'c')은 자동으로 삭제됩니다.

t = ('A', 'b', 'c')

077
다음 튜플을 리스트로 변환하라.

interest = ('삼성전자', 'LG전자', 'SK Hynix')
정답확인
data = list(interest)

078
다음 리스트를 튜플로 변경하라.

interest = ['삼성전자', 'LG전자', 'SK Hynix']
정답확인
data = tuple(interest)
---
079 튜플 언팩킹
다음 코드의 실행 결과를 예상하라.

temp = ('apple', 'banana', 'cake')
a, b, c = temp
print(a, b, c)
정답확인
apple banana cake
---
080 range 함수
1 부터 99까지의 정수 중 짝수만 저장된 튜플을 생성하라.

(2, 4, 6, 8 ... 98)
정답확인
data = **tuple(range(2,** 100, 2))
print(data)
---

081 별 표현식
기본적으로 데이터 언패킹은 좌변의 변수와 우변 데이터 개수가 같아야 합니다. 하지만 star expression을 사용하면 변수의 개수가 달라도 데이터 언패킹을 할 수 있습니다. 튜플에 저장된 데이터 중에서 앞에 있는 두 개의 데이터만 필요할 경우 나머지 데이터의 언패킹 코드를 작성할 필요가 없습니다.

```
>> a, b, *c = (0, 1, 2, 3, 4, 5)
>> a
0
>> b
1
>> c
[2, 3, 4, 5]
```

다음과 같이 10개의 값이 저장된 scores 리스트가 있을 때, start expression을 사용하여 좌측 8개의 값을 valid_score 변수에 바인딩하여라.

```
scores = [8.8, 8.9, 8.7, 9.2, 9.3, 9.7, 9.9, 9.5, 7.8, 9.4]
```

정답확인

```
scores = [8.8, 8.9, 8.7, 9.2, 9.3, 9.7, 9.9, 9.5, 7.8, 9.4]
*valid_score, _, _= scores
print(valid_score)
---------------------------------------------
[8.8, 8.9, 8.7, 9.2, 9.3, 9.7, 9.9, 9.5]
=============================================
```

082
다음과 같이 10개의 값이 저장된 scores 리스트가 있을 때, start expression을 사용하여 우측 8개의 값을 valid_score 변수에 바인딩하여라.

```
scores = [8.8, 8.9, 8.7, 9.2, 9.3, 9.7, 9.9, 9.5, 7.8, 9.4]
```

정답확인

```
scores = [8.8, 8.9, 8.7, 9.2, 9.3, 9.7, 9.9, 9.5, 7.8, 9.4]
a, b, *valid_score = scores
print(valid_score)
---------------------------------------
```

083
다음과 같이 10개의 값이 저장된 scores 리스트가 있을 때, start expression을 사용하여 가운데 있는 8개의 값을 valid_score 변수에 바인딩하여라.

```
scores = [8.8, 8.9, 8.7, 9.2, 9.3, 9.7, 9.9, 9.5, 7.8, 9.4]
```

정답확인

```
scores = [8.8, 8.9, 8.7, 9.2, 9.3, 9.7, 9.9, 9.5, 7.8, 9.4]
```

a, *valid_score, b = scores
print(valid_score)
---
084 비어있는 딕셔너리
temp 이름의 비어있는 딕셔너리를 만들라.

정답확인
temp = { }

085
다음 아이스크림 이름과 희망 가격을 딕셔너리로 구성하라.

이름 희망 가격
메로나 1000
폴라포 1200
빵빠레 1800
정답확인
ice = {"메로나": 1000, "폴라포": 1200, "빵빠레": 1800}
print(ice)

086
085 번의 딕셔너리에 아래 아이스크림 가격정보를 추가하라.

이름 희망 가격
죠스바 1200
월드콘 1500
정답확인
ice = {"메로나": 1000, "폴라포": 1200, "빵빠레": 1800}
ice["죠스바"] = 1200
ice["월드콘"] = 1500
print(ice)

{'메로나': 1000, '폴라포': 1200, '빵빠레': 1800, '죠스바': 1200, '월드콘': 1500}
==========
087
다음 딕셔너리를 사용하여 메로나 가격을 출력하라.

```
ice = {'메로나': 1000,
       '폴로포': 1200,
       '빵빠레': 1800,
       '죠스바': 1200,
```

```
        '월드콘': 1500}
```
실행 예:
메로나 가격: 1000
정답확인
```
print("메로나 가격: ", ice["메로나"])
```

088
다음 딕셔너리에서 메로나의 가격을 1300으로 수정하라.

```
ice = {'메로나': 1000,
        '폴로포': 1200,
        '빵빠레': 1800,
        '죠스바': 1200,
        '월드콘': 1500}
```
정답확인
```
ice["메로나"] = 1300
```

089
다음 딕셔너리에서 메로나를 삭제하라.

```
ice = {'메로나': 1000,
        '폴로포': 1200,
        '빵빠레': 1800,
        '죠스바': 1200,
        '월드콘': 1500}
```
정답확인
```
del ice["메로나"]
print(ice)
```

090
다음 코드에서 에러가 발생한 원인을 설명하라.

```
>> icecream = {'폴라포': 1200, '빵빠레': 1800, '월드콘': 1500, '메로나': 1000}
>> icecream['누가바']
Traceback (most recent call last):
  File "<pyshell#69>", line 1, in <module>
    icecream['누가바']
KeyError: '누가바'
```
정답확인
딕셔너리에 없는 키를 사용해서 인덱싱하면 에러가 발생합니다.
---

091 딕셔너리 생성
아래의 표에서, 아이스크림 이름을 키값으로, (가격, 재고) 리스트를 딕셔너리의 값으로 저장하라. 딕셔너리의 이름은 inventory로 한다.

이름	가격	재고
메로나	300	20
비비빅	400	3
죠스바	250	100

정답확인
```
inventory = {"메로나": [300, 20],
             "비비빅": [400, 3],
             "죠스바": [250, 100]}
print(inventory)
```
---
092 딕셔너리 인덱싱
inventory 딕셔너리에서 메로나의 가격을 화면에 출력하라.

```
inventory = {"메로나": [300, 20],
             "비비빅": [400, 3],
             "죠스바": [250, 100]}
```
실행 예시:
300 원
정답확인
```
print(inventory["메로나"][0], "원")
```
---
093 딕셔너리 인덱싱
inventory 딕셔너리에서 메로나의 재고를 화면에 출력하라.

```
inventory = {"메로나": [300, 20],
             "비비빅": [400, 3],
             "죠스바": [250, 100]}
```
실행 예시:
20 개
정답확인
```
print(inventory["메로나"][1], "개")
```
---
094 딕셔너리 추가
inventory 딕셔너리에 아래 데이터를 추가하라.

```
inventory = {"메로나": [300, 20],
             "비비빅": [400, 3],
```

```
                "죠스바": [250, 100]}
이름      가격      재고
월드콘   500       7
```
실행 예시:
```
>> print(inventory)
{'메로나': [300, 20], '비비빅': [400, 3], '죠스바': [250, 100], '월드콘': [500, 7]}
```
정답확인
```
inventory = {"메로나": [300, 20],
             "비비빅": [400, 3],
             "죠스바": [250, 100]}
inventory["월드콘"] = [500, 7]
print(inventory)
```
---

095 딕셔너리 keys() 메서드

다음의 딕셔너리로부터 key 값으로만 구성된 리스트를 생성하라.

```
icecream = {'탱크보이': 1200, '폴라포': 1200, '빵빠레': 1800, '월드콘': 1500, '메로나': 1000}
```
정답확인
```
icecream = {'탱크보이': 1200, '폴라포': 1200, '빵빠레': 1800, '월드콘': 1500, '메로나': 1000}
ice = list(icecream.keys())
print(ice)
```

```
['탱크보이', '폴라포', '빵빠레', '월드콘', '메로나']
```
==========

096 딕셔너리 values() 메서드

다음의 딕셔너리에서 values 값으로만 구성된 리스트를 생성하라.

```
icecream = {'탱크보이': 1200, '폴라포': 1200, '빵빠레': 1800, '월드콘': 1500, '메로나': 1000}
```
정답확인
```
icecream = {'탱크보이': 1200, '폴라포': 1200, '빵빠레': 1800, '월드콘': 1500, '메로나': 1000}
price = list(icecream.values())
print(price)
```

```
[1200, 1200, 1800, 1500, 1000]
```
==========

097 딕셔너리 values() 메서드

icecream 딕셔너리에서 아이스크림 판매 금액의 총합을 출력하라.

icecream = {'탱크보이': 1200, '폴라포': 1200, '빵빠레': 1800, '월드콘': 1500, '메로나': 1000}
출력 예시:
6700
정답확인
icecream = {'탱크보이': 1200, '폴라포': 1200, '빵빠레': 1800, '월드콘': 1500, '메로나': 1000}
print(sum(icecream.values()))
---
098 딕셔너리 update 메서드
아래의 new_product 딕셔너리를 다음 icecream 딕셔너리에 추가하라.

icecream = {'탱크보이': 1200, '폴라포': 1200, '빵빠레': 1800, '월드콘': 1500, '메로나': 1000}
new_product = {'팥빙수':2700, '아맛나':1000}
실행 예시:
>> print(icecream)
{'탱크보이': 1200, '폴라포': 1200, '빵빠레': 1800, '월드콘': 1500, '메로나': 1000, '팥빙수':2700, '아맛나':1000}
정답확인
icecream = {'탱크보이': 1200, '폴라포': 1200, '빵빠레': 1800, '월드콘': 1500, '메로나': 1000}
new_product = {'팥빙수':2700, '아맛나':1000}
icecream.update(new_product)
print(icecream)
---
099 zip과 dict
아래 두 개의 튜플을 하나의 딕셔너리로 변환하라. keys를 키로, vals를 값으로 result 이름의 딕셔너리로 저장한다.

keys = ("apple", "pear", "peach")
vals = (300, 250, 400)
실행 예시:
>> print(result)
{'apple': 300, 'pear': 250, 'peach': 400}
정답확인
keys = ("apple", "pear", "peach")
vals = (300, 250, 400)
result = dict(zip(keys, vals))
print(result)

---
100 zip과 dict
date와 close_price 두 개의 리스트를 close_table 이름의 딕셔너리로 생성하라.

date = ['09/05', '09/06', '09/07', '09/08', '09/09']
close_price = [10500, 10300, 10100, 10800, 11000]
실행 예시:
>> print(close_table)
{'09/05': 10500, '09/06': 10300, '09/07': 10100, '09/08': 10800, '09/09': 11000}
정답확인
date = ['09/05', '09/06', '09/07', '09/08', '09/09']
close_price = [10500, 10300, 10100, 10800, 11000]
close_table = dict(zip(date, close_price))
print(close_table)
==========
101
파이썬에서 True 혹은 False를 갖는 데이터 타입은 무엇인가?

정답확인
bool 타입입니다.

102
아래 코드의 출력 결과를 예상하라

print(3 == 5)
정답확인
False

103
아래 코드의 출력 결과를 예상하라

print(3 < 5)
정답확인
True

104
아래 코드의 결과를 예상하라.

x = 4
print(1 < x < 5)
정답확인

True

105
아래 코드의 결과를 예상하라.

```
print ((3 == 3) and (4 != 3))
```

정답확인
True

106
아래 코드에서 에러가 발생하는 원인에 대해 설명하라.

```
print(3 => 4)
```

정답확인
지원하지 않는 연산자입니다.

107
아래 코드의 출력 결과를 예상하라

```
if 4 < 3:
    print("Hello World")
```

정답확인
조건을 만족하지 않기 때문에 아무 결과도 출력되지 않습니다.

108
아래 코드의 출력 결과를 예상하라

```
if 4 < 3:
    print("Hello World.")
else:
    print("Hi, there.")
```

정답확인
조건을 만족하지 않기 때문에 else에서 들여쓰기 된 문장이 실행됩니다.

Hi, there.

109
아래 코드의 출력 결과를 예상하라

```
if True :
    print ("1")
```

```
    print ("2")
else :
    print("3")
print("4")
```
정답확인
1
2
4

110
아래 코드의 출력 결과를 예상하라

```
if True :
    if False:
        print("1")
        print("2")
    else:
        print("3")
else :
    print("4")
print("5")
```
정답확인
3
5

111
사용자로부터 입력받은 문자열을 두 번 출력하라. 아래는 사용자가 "안녕하세요"를 입력한 경우의 출력 결과이다.

>> 안녕하세요
안녕하세요안녕하세요
정답확인
```
user = input("입력:")
print(user * 2)
```

112
사용자로부터 하나의 숫자를 입력받고, 입력 받은 숫자에 10을 더해 출력하라.

>> 숫자를 입력하세요: 30
40
정답확인

```
user = input("숫자를 입력하세요: ")
print(10 + int(user))
print(user)
```

숫자를 입력하세요: >? 333
343
333
==========
113
사용자로부터 하나의 숫자를 입력 받고 짝수/홀수를 판별하라.

>> 30
짝수
정답확인
```
user = input("")
if int(user) % 2 == 0:
    print("짝수")
else:
    print("홀수")
```

114
사용자로부터 값을 입력받은 후 해당 값에 20을 더한 값을 출력하라. 단 사용자가 입력한 값과 20을 더한 계산 값이 255를 초과하는 경우 255를 출력해야 한다.

>> 입력값: 200
출력값: 220
>> 입력값: 240
출력값: 255
정답확인
```
user = input("입력값: ")
num = 20 + int(user)
if num > 255:
    print(255)
else:
    print(num)
```

115
사용자로부터 하나의 값을 입력받은 후 해당 값에 20을 뺀 값을 출력하라. 단 출력 값의 범위는 0~255이다. 예를 들어 결괏값이 0보다 작은 값이 되는 경우 0을 출력하고 255보다 큰 값이 되는 경우 255를 출력해야 한다.

```
>> 입력값: 200
출력값: 180
>> 입력값: 15
출력값: 0
정답확인
user = input("입력값: ")
num = int(user) - 20
if num > 255:
    print(255)
elif num < 0:
    print(0)
else:
    print(num)
```

116

사용자로부터 입력 받은 시간이 정각인지 판별하라.

```
>> 현재시간:02:00
정각 입니다.
>> 현재시간:03:10
정각이 아닙니다
정답확인
time = input("현재시간: ")
if time[-2:] == "00":
    print("정각 입니다.")
else:
    print("정각이 아닙니다.")
```

117

사용자로 입력받은 단어가 아래 fruit 리스트에 포함되어 있는지를 확인하라. 포함되었다면 "정답입니다"를 아닐 경우 "오답입니다" 출력하라.

```
fruit = ["사과", "포도", "홍시"]
>> 좋아하는 과일은? 사과
정답입니다.
정답확인
fruit = ["사과", "포도", "홍시"]
user = input("좋아하는 과일은?")
if user in fruit:
    print("정답입니다.")
else:
```

```
    print("오답입니다.")
```

118
투자 경고 종목 리스트가 있을 때 사용자로부터 종목명을 입력 받은 후 해당 종목이 투자 경고 종목이라면 '투자 경고 종목입니다'를 아니면 "투자 경고 종목이 아닙니다."를 출력하는 프로그램을 작성하라.

```
warn_investment_list = ["Microsoft", "Google", "Naver", "Kakao", "SAMSUNG", "LG"]
```
정답확인
```
warn_investment_list = ["Microsoft", "Google", "Naver", "Kakao", "SAMSUNG", "LG"]
종목 = input("종목: ")
if 종목 in warn_investment_list:
    print("투자 경고 종목입니다.")
else:
    print("투자 경고 종목이 아닙니다.")
```

119
아래와 같이 fruit 딕셔너리가 정의되어 있다. 사용자가 입력한 값이 딕셔너리 키 (key) 값에 포함되었다면 "정답입니다"를 아닐 경우 "오답입니다" 출력하라.

```
fruit = {"봄" : "딸기", "여름" : "토마토", "가을" : "사과"}
>> 제가 좋아하는 계절은: 봄
정답입니다.
```
정답확인
```
fruit = {"봄" : "딸기", "여름" : "토마토", "가을" : "사과"}
user = input("제가 좋아하는 계절은: ")
if user in fruit:
    print("정답입니다.")
else:
    print("오답입니다.")
```

120
아래와 같이 fruit 딕셔너리가 정의되어 있다. 사용자가 입력한 값이 딕셔너리 값 (value)에 포함되었다면 "정답입니다"를 아닐 경우 "오답입니다" 출력하라.

```
fruit = {"봄" : "딸기", "여름" : "토마토", "가을" : "사과"}
>> 좋아하는 과일은? 한라봉
오답입니다.
```
정답확인
```
fruit = {"봄" : "딸기", "여름" : "토마토", "가을" : "사과"}
user = input("좋아하는 과일은?")
```

```
if user in fruit.values():
    print("정답입니다.")
else:
    print("오답입니다.")
```

121

사용자로부터 문자 한 개를 입력 받고, 소문자일 경우 대문자로, 대문자 일 경우, 소문자로 변경해서 출력하라.

```
>> a
A
```

힌트-1 : islower() 함수는 문자의 소문자 여부를 판별합니다. 만약 소문자일 경우 True, 대문자일 경우 False를 반환합니다. 힌트-2 : upper() 함수는 대문자로, lower() 함수는 소문자로 변경합니다.

정답확인

```
user = input("")
if user.islower():
    print(user.upper())
else:
    print(user.lower())
```

122

점수 구간에 해당하는 학점이 아래와 같이 정의되어 있다. 사용자로부터 score를 입력받아 학점을 출력하라.

점수	학점
81~100	A
61~80	B
41~60	C
21~40	D
0~20	E

```
>> score: 83
grade is A
```

정답확인

```
score = input("score: ")
score = int(score)
if 81 <= score <= 100:
    print("grade is A")
elif 61 <= score <= 80:
    print("grade is B")
```

```
elif 41 <= score <= 60:
    print("grade is C")
elif 21 <= score <= 40:
    print("grade is D")
else:
    print("grade is E")
```

123

사용자로부터 달러, 엔, 유로, 또는 위안 금액을 입력받은 후 이를 원으로 변환하는 프로그램을 작성하라. 각 통화별 환율은 다음과 같다. 사용자는 100 달러, 1000 엔, 13 유로, 100 위안과 같이 금액과 통화명 사이에 공백을 넣어 입력한다고 가정한다.

통화명	환율
달러	1167
엔	1.096
유로	1268
위안	171

```
>> 입력: 100 달러
116700.00 원
```

정답확인

```
환율 = {"달러": 1167,
        "엔": 1.096,
        "유로": 1268,
        "위안": 171}
user = input("입력: ")
num, currency = user.split()
print(float(num) * 환율[currency], "원")
```

124

사용자로부터 세 개의 숫자를 입력 받은 후 가장 큰 숫자를 출력하라.

```
>> input number1: 10
>> input number2: 9
>> input number3: 20
20
```

정답확인

```
num1 = input("input number1: ")
num2 = input("input number2: ")
num3 = input("input number3: ")
num1 = int(num1)
num2 = int(num2)
```

```
num3 = int(num3)

if num1 > num2 and num1 > num3:
    print(num1)
elif num2 > num1 and num2 > num3:
    print(num2)
else:
    print(num3)
```

125

휴대폰 번호 앞자리에 따라 통신사는 아래와 같이 구분된다. 사용자로부터 휴대전화 번호를 입력 받고, 통신사를 출력하는 프로그램을 작성하라.

번호	통신사
011	SKT
016	KT
019	LGU
010	알수없음

```
>> 휴대전화 번호 입력: 011-345-1922
당신은 SKT 사용자입니다.
```

정답확인

```
number = input("휴대전화 번호 입력: ")
num = number.split("-")[0]
if num == "011":
    com = "SKT"
elif num == "016":
    com = "KT"
elif num == "019":
    com = "LGU"
else:
    com = "알수없음"
print(f"당신은 {com} 사용자입니다.")
```

126

우편번호는 5자리로 구성되는데, 앞의 세자리는 구를 나타낸다. 예를 들어, 강북구의 경우 010, 011, 012 세 자리로 시작한다.

-	0	1	2	3	4	5	6	7	8	9
01	강북구	강북구	강북구	도봉구	도봉구	도봉구	노원구	노원구	노원구	노 원구

사용자로 부터 5자리 우편번호를 입력받고 구를 판별하라

```
>> 우편번호: 01400
도봉구
정답확인
우편번호 = input("우편번호: ")
우편번호 = 우편번호[:3]
if 우편번호 in ["010", "011", "012"]:
    print("강북구")
elif 우편번호 in ["014", "015", "016"]:
    print("도봉구")
else:
    print("노원구")
```

127

주민등록번호 뒷 자리 7자리 중 첫째 자리는 성별을 나타내는데, 1, 3은 남자 2, 4는 여자를 의미한다. 사용자로부터 13자리의 주민등록번호를 입력 받은 후 성별 (남자, 여자)를 출력하는 프로그램을 작성하라.

```
>> 주민등록번호: 821010-1635210
남자
정답확인
주민번호 = input("주민등록번호: ")
주민번호 = 주민번호.split("-")[1]
if 주민번호[0] == "1" or 주민번호[0] == "3":
    print("남자")
else:
    print("여자")
```

128

주민등록번호의 뒷 자리 7자리 중 두번째와 세번째는 지역코드를 의미한다. 주민 등록 번호를 입력 받은 후 출생지가 서울인지 아닌지 판단하는 코드를 작성하라

```
지역코드출생지
00 ~ 08 서울
09 ~ 12 부산
>> 주민등록번호: 821010-1635210
서울이 아닙니다.
>> 주민등록번호: 861010-1015210
서울 입니다.
정답확인
주민번호 = input("주민등록번호: ")
```

```
뒷자리 = 주민번호.split("-")[1]
if 0 <= int(뒷자리[1:3]) <= 8:
    print("서울입니다.")
else:
    print("서울이 아닙니다.")
```

129

주민등록번호는 13자리로 구성되는데 마지막 자리수는 주민등록번호의 유효성을 체크하는데 사용된다. 먼저 앞에서부터 12자리의 숫자에 2, 3, 4, 5, 6, 7, 8, 9, 2, 3, 4, 5를 차례로 곱한 뒤 그 값을 전부 더한다. 연산 결과 값을 11로 나누면 나머지가 나오는데 11에서 나머지를 뺀 값이 주민등록번호의 마지막 번호가 된다.

```
  8 2 1 0 1 0 - 1 6 3 5 2 1 0
x 2 3 4 5 6 7   8 9 2 3 4 5
-----------------------------
```

1차 계산: (8*2 + 2*3 + 1*4 + 0*5 + 1*6 + 0*7 + 1*8 + 6*9 + 3*2 + 5*3 + 2*4 + 1*5) = (128 % 11) = 7

2차 계산: 11 -7 = 4

위와 같이 821010-1635210에 대해서 계산을 해보면 마지막 자리는 4가 되어야 함을 알 수 있다. 즉, 821010-1635210은 유효하지 않은 주민등록번호임을 알 수 있다.

다음과 같이 사용자로부터 주민등록번호를 입력받은 후 주민등록번호가 유효한지를 출력하는 프로그램을 작성하라.

```
>> 주민등록번호: 821010-1635210
유효하지 않은 주민등록번호입니다.
정답확인
num = input("주민등록번호: ")
계산1 = int(num[0]) * 2 + int(num[1]) * 3 + int(num[2]) * 4 + int(num[3]) * 5 + int(num[4]) * 6 + \
        int(num[5]) * 7 + int(num[7]) * 8 + int(num[8]) * 9 + int(num[9]) * 2 + int(num[10])* 3 + \
        int(num[11])* 4 + int(num[12]) * 5
계산2 = 11 - (계산1 % 11)
계산3 = str(계산2)

if num[-1] == 계산3[-1]:
    print("유효한 주민등록번호입니다.")
else:
    print("유효하지 않은 주민등록번호입니다.")
```

130

아래 코드는 비트코인의 가격 정보를 딕셔너리로 가져오는 코드이다.

```
import requests
btc = requests.get("https://api.bithumb.com/public/ticker/").json()['data']
```

btc 딕셔너리 안에는 시가, 종가, 최고가, 최저가 등이 저장되어 있다. 최고가와 최저가의 차이를 변동폭으로 정의할 때 (시가 + 변동폭)이 최고가 보다 높을 경우 "상승장", 그렇지 않은 경우 "하락장" 문자열을 출력하라.

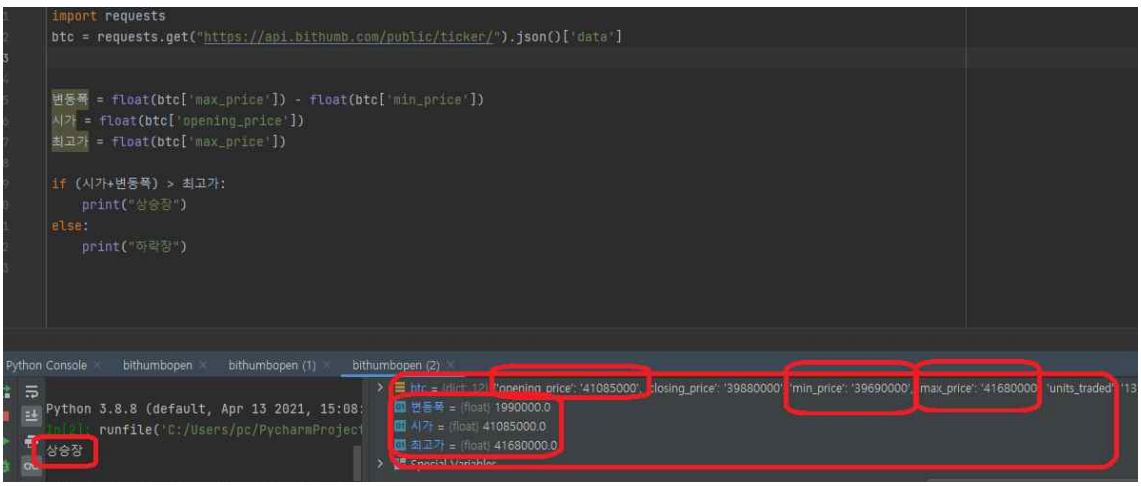
Key Name	Description
opening_price	최근 24시간 내 시작 거래금액
closing_price	최근 24시간 내 마지막 거래금액
min_price	최근 24시간 내 최저 거래금액
max_price	최근 24시간 내 최고 거래금액

정답확인

```
import requests
btc = requests.get("https://api.bithumb.com/public/ticker/").json()['data']

변동폭 = float(btc['max_price']) - float(btc['min_price'])
시가 = float(btc['opening_price'])
최고가 = float(btc['max_price'])

if (시가+변동폭) > 최고가:
    print("상승장")
else:
    print("하락장")
```



131

for문의 실행결과를 예측하라.

```
과일 = ["사과", "귤", "수박"]
for 변수 in 과일:
    print(변수)
```

정답확인

리스트에 들어있는 문자열이 한 라인에 하나씩 출력됩니다.

사과
귤
수박

132

for문의 실행결과를 예측하라.

```
과일 = ["사과", "귤", "수박"]
for 변수 in 과일:
  print("#####")
```

정답확인

기초문법 배우기-1에서 for문의 핵심은 "들여쓰기된 코드가 자료구조에 저장된 데이터 개수만큼 반복된다"라고 설명했습니다. 과일 = ["사과", "귤", "수박"] 에는 세 개의 데이터가 저장돼 있으므로 들여쓰기된 print("####")코드가 세 번 실행됩니다.

#####
#####
#####

133

다음 for 문과 동일한 기능을 수행하는 코드를 작성하세요.

```
for 변수 in ["A", "B", "C"]:
  print(변수)
```

정답확인

파이썬 인터프리터가 어떤 순서로 코드를 실행하는지를 떠올려 보세요. "변수 바인딩(라인 1) -> 변수 출력 (라인 2)" 과정을 자료구조 데이터 개수 만큼 반복합니다.

```
변수 = "A"
print(변수)
변수 = "B"
print(변수)
변수 = "C"
print(변수)
```

간단해서 변수의 바인딩없이 다음과 같이 코드를 작성해도 됩니다. 모두 동일한 결과를 출력

합니다.

```
print("A")
print("B")
print("C")
```

134
for문을 풀어서 동일한 동작을하는 코드를 작성하라.

```
for 변수 in ["A", "B", "C"]:
  print("출력:", 변수)
```
정답확인
```
변수 = "A"
print("출력:", 변수)
변수 = "B"
print("출력:", 변수)
변수 = "C"
print("출력:", 변수)
print("출력:", "A")
print("출력:", "B")
print("출력:", "C")
```

135
for문을 풀어서 동일한 동작을 하는 코드를 작성하라.

```
for 변수 in ["A", "B", "C"]:
  b = 변수.lower()
  print("변환:", b)
```
정답확인
들여쓰기된 코드의 내용은 중요하지 않습니다. 파이썬 인터프리터가 코드를 해석하는 순서를 익히는 것이 중요합니다. 라인1의 for문이 실행되면 변수에 첫 번재 데이터를 바인딩합니다. 그리고 들여쓰기된 라인 2의 코드를 실행합니다.
```
변수 = "A"
b = 변수.lower()
print("변환:", b)
```
들여쓰기된 코드의 실행이 끝나면 라인 1로 이동해서 변수에 두 번재 데이터를 바인딩합니다. 그리고 들여쓰기된 코드를 실행합니다.

```
변수 = "B"
b = 변수.lower()
print("변환:", b)
```

다음으로 변수 "C"를 바인딩하고 출력하면 되겠죠? for문을 풀어쓴 전체 코드는 다음과 같습니다.

```
변수 = "A"
b = 변수.lower()
print("변환:", b)
변수 = "B"
b = 변수.lower()
print("변환:", b)
변수 = "C"
b = 변수.lower()
print("변환:", b)
```

136

다음 코드를 for문으로 작성하라.

```
변수 = 10
print(변수)
변수 = 20
print(변수)
변수 = 30
print(변수)
```

정답확인

for문은 동일한 루틴을 반복해서 실행하는 문법이었습니다. 우선 문제에서 변하지 않고 반복되는 코드를 찾아보세요. print(변수) 부분이죠? 변하는 부분은 변수에 값을 대입하는 코드입니다.

```
변수 = 10
print(변수)
변수 = 20
print(변수)
변수 = 30
print(변수)
```

반복되는 코드는 들여쓰기로 표현하고 for문에서 변하는 부분을 기술해 줍니다.

```
for 변수 in [10, 20, 30]:
  print(변수)
```

다음과 같이 리스트 변수에 데이터를 바인딩하고, 변수를 for문의 자료구조 자리에 넣어줘도 됩니다.

```
리스트 = [10, 20, 30]
```

```
for 변수 in 리스트:
  print(변수)
```

138
다음 코드를 for문으로 작성하라.

```
print(10)
print("-------")
print(20)
print("-------")
print(30)
print("-------")
```
정답확인
변하는 값을 변수에 바인딩하고, 변수를 사용해서 나머지 코드를 작성합니다.
```
변수 = 10
print(변수)
print("-------")

변수 = 20
print(변수)
print("-------")

변수 = 30
print(변수)
print("-------")
```
변하는 부분은 for문과 함께 리스트로 처리하고 변하지 않는 코드는 들여쓰기와 함께 적어줍니다.

```
for 변수 in [10, 20, 30]:
  print(변수)
  print("-------")
```

139
다음 코드를 for문으로 작성하라.

```
print("++++")
print(10)
print(20)
print(30)
```
정답확인
print("++++") 코드가 함정입니다. 파이썬 for문은 반복되는 코드에만 사용합니다.

```
print("++++")
for 변수 in [10, 20, 30]:
  print(변수)
```

140
다음 코드를 for문으로 작성하라.

```
print("-------")
print("-------")
print("-------")
print("-------")
```
정답확인
네 번 반복 출력하는 코드를 작성하는 문제입니다.
- 변하는 값 : 없음
- 변하지 않는 코드 : print("-------")
지금까지 정리한 내용을 코드로 작성하면 아래와 같습니다. "????" 부분을 어떻게 채워야 할까요?

```
for 변수 in ?????:
  print("-------")
```
파이썬 문법에서 강조했던 내용을 떠올려 봅시다.

파이썬 for문은 들여쓰기된 코드가 자료구조의 데이터 개수만큼 반복된다는 사실이 중요합니다.

문제에서 원하는 것은 들여쓰기된 코드가 4번 실행되는 것입니다. 그렇다면 for문의 자료구조 자리에 데이터가 4개저장된 리스트가 들어가면 어떻게 될까요?

```
for 변수 in ["가", "나", "다", "라"]:
  print("-------")
```
변수에 값이 바인딩되지만 사용되지 않을뿐 실행해보면 들여쓰기된 코드가 4번 실행됩니다. 위 예제에서는 자료구조에 "가나다라" 문자열 값을 넣었지만 아래와 같이 1,2,3,4 숫자 값을 넣어도 동일한 동작을 합니다. 자료구조에 네 개의 데이터가 들어있으면 됩니다.

```
for 변수 in [1, 2, 3, 4]:
  print("-------")
```

141
다음과 같이 판매가가 저장된 리스트가 있을 때 부가세가 포함된 가격을 for 문을 사용해서 화면에 출력하라. 단 부가세는 10원으로 가정한다.

리스트 = [100, 200, 300]
110
210
310
정답확인
우선, for문을 사용하지 않고 모든 값에 10을 더하는 코드를 작성해 봅시다.
변수 = 100
print(변수 + 10)
변수 = 200
print(변수 + 10)
변수 = 300
print(변수 + 10)
변하는 값은 for문의 자료구조 자리에, 변하지 않는 코드는 들여쓰기 후에 작성합니다.

리스트 = [100, 200, 300]
for 변수 in 리스트:
  print(변수 + 10)
파이썬 인터프리터가 코드를 실행하는 순서를 익혀서, 한 번에 for문으로 작성할 수 있어야합니다.
(100 바인딩 - 들여쓰기 코드 실행)
(200 바인딩 - 들여쓰기 코드 실행)
(300 바인딩 - 들여쓰기 코드 실행)

142
for 문을 사용해서 리스트에 저장된 값을 다음과 같이 출력하라.

리스트 = ["김밥", "라면", "튀김"]
오늘의 메뉴: 김밥
오늘의 메뉴: 라면
오늘의 메뉴: 튀김
정답확인
for문을 사용하지 않고 문자열을 출력하는 코드를 작성해 봅시다. 011번 문제와는 리스트안의 데이터 타입이 문자열이라는 것이 차이점입니다. 문자열의 덧셈은 두 문자열을 이어붙입니다.
메뉴 = "김밥"
print("오늘의 메뉴: " + 메뉴)
메뉴 = "라면"
print("오늘의 메뉴: " + 메뉴)
메뉴 = "튀김"
print("오늘의 메뉴: " + 메뉴)
변하는 값은 for문의 자료구조 자리에, 변하지 않는 코드는 들여쓰기 후에 작성합니다.

```
리스트 = ["김밥", "라면", "튀김"]
for 메뉴 in 리스트:
  print("오늘의 메뉴: " + 메뉴)
```

문자열 덧셈 대신 print() 함수의 기능을 사용해서 다음과 같이 코드를 작성해도 좋습니다.

print() 함수는 입력한 값을 한칸의 공백과 함께 이어붙여 출력합니다.

```
리스트 = ["김밥", "라면", "튀김"]
for 메뉴 in 리스트:
  print("오늘의 메뉴:", 메뉴)
```

143

리스트에 주식 종목이름이 저장돼 있다.

```
리스트 = ["SK하이닉스", "삼성전자", "LG전자"]
```

저장된 문자열의 길이를 다음과 같이 출력하라.

```
6
4
4
```

정답확인

문자열의 길이를 구하는 함수 len()을 사용합니다.

```
리스트 = ["SK하이닉스", "삼성전자", "LG전자"]
for 종목명 in 리스트:
  길이 = len(종목명)
  print(길이)
```

간단하기 때문에 한줄로 표현해도 좋습니다.

```
리스트 = ["SK하이닉스", "삼성전자", "LG전자"]
for 종목명 in 리스트:
  print(len(종목명))
```

한 번에 코드를 작성하기 어렵다면 for문을 사용하지 않고 풀어서 코드를 작성해보세요.

144

리스트에는 동물이름이 문자열로 저장돼 있다.

```
리스트 = ['dog', 'cat', 'parrot']
```

동물 이름과 글자수를 다음과 같이 출력하라.

```
dog 3
cat 3
```

parrot 6
정답확인
리스트에는 동물이름이 문자열로 저장돼 있다.
```
리스트 = ['dog', 'cat', 'parrot']
for 이름 in 리스트:
  print(이름, len(이름))
```

145
리스트에 동물 이름 저장돼 있다.

```
리스트 = ['dog', 'cat', 'parrot']
```
for문을 사용해서 동물 이름의 첫 글자만 출력하라.

d
c
p
정답확인
어렵게 느껴진다면 for문을 사용하지 않고 코드를 작성해봅시다. 바인딩된 문자열의 인덱싱을 사용해서 첫 글자를 화면에 출력합니다.
```
이름 = "dog"
print(이름[0])
변수 = "cat"
print(이름[0])
변수 = "parrot"
print(이름[0])
```
변하는 값은 for문의 자료구조 자리에, 변하지 않는 코드는 들여쓰기 후에 작성합니다. 쉽죠?

```
리스트 = ['dog', 'cat', 'parrot']
for 이름 in 리스트:
  print(이름[0])
```

146
리스트에는 세 개의 숫자가 바인딩돼 있다.

```
리스트 = [1, 2, 3]
```
for문을 사용해서 다음과 같이 출력하라.

3 x 1
3 x 2
3 x 3
정답확인

012 문제를 풀었다면 충분히 응용할 수 있어야합니다. "3 x " 문자열과 숫자를 이어붙여 출력하면됩니다. 다만 변수에 바인딩된 값이 숫자이기 때문에 문자로 변환 후에 문자열 연산을 사용할 수 있습니다.

```
리스트 = [1, 2, 3]
for 변수 in 리스트:
  print("3 x " + str(변수))
```

혹은 print() 함수가 제공하는 기능을 사용할 수 있습니다.

```
리스트 = [1, 2, 3]
for 변수 in 리스트:
  print("3 x", 변수)
```

147

리스트에는 세 개의 숫자가 바인딩돼 있다.

```
리스트 = [1, 2, 3]
```

for문을 사용해서 다음과 같이 출력하라.

```
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
```

정답확인

어려워 보인다면, for문을 사용하지 않고 풀어서 코드를 작성해야 합니다. 핵심은 print() 함수에서 3 * 변수 부분입니다. 변수를 활용해서 원하는 값을 만들어 냈습니다.

```
변수 = 1
print("3 x ", 변수, "=", 3 * 변수)
변수 = 2
print("3 x ", 변수, "=", 3 * 변수)
변수 = 2
print("3 x ", 변수, "=", 3 * 변수)
```

값만 변하고 있기 때문에 쉽게 for문으로 작성할 수 있습니다.

```
리스트 = [1, 2, 3]
for 변수 in 리스트:
  print("3 x ", 변수, "=", 3 * 변수)
```

1장에서 배운 문자열 함수를 활용하면 코드가 더욱 읽기 쉬워집니다.

```
리스트 = [1, 2, 3]
for 변수 in 리스트:
  print("3 x {} = {}".format(변수, 3 * 변수))
```

148
리스트에는 네 개의 문자열이 바인딩돼 있다.

리스트 = ["가", "나", "다", "라"]
for문을 사용해서 다음과 같이 출력하라.

나
다
라
정답확인
리스트의 슬라이싱을 사용해서 일부 데이터를 "나다라" 변수에 바인딩합니다. "나다라"에 바인딩 된 리스트로 for문을 사용하면 원하는 결과를 얻을 수 있습니다. [1:] 코드가 어색하다면 2장을 복습합시다.

```
리스트 = ["가", "나", "다", "라"]
나다라 = 리스트[1:]
for 변수 in 나다라:
  print(변수)
```

리스트를 슬라이싱한 결과를 "리스트" 변수에 업데이트 할 수도있습니다. 전체 값을 출력하는 코드에서 for문의 코드 변경없이 추가 코드만 한줄 작성합니다.

```
리스트 = ["가", "나", "다", "라"]
리스트 = 리스트[1:]
for 변수 in 리스트:
  print(변수)
```

간단하니 for문 자료구조 자리에 슬라이싱 코드를 한번에 작성할 수도있습니다.

```
리스트 = ["가", "나", "다", "라"]
for 변수 in 리스트[1:]:
  print(변수)
```

세 개의 코드는 모두 동일한 동작을 하는데, 되도록이면 간단하게 쓰는게 좋겠죠? 세 번째 방법을 눈에 익혀 두세요.

149
리스트에는 네 개의 문자열이 바인딩돼 있다.

리스트 = ["가", "나", "다", "라"]
for문을 사용해서 다음과 같이 출력하라.

가
다
정답확인

리스트 슬라이싱에서 [시작:끝:증감폭] 세번째 값은 값을 가져오는 단위를 의미했습니다. 증감폭을 2로 설정해서 "가"와 "다" 값만 슬라이싱합니다.

```
리스트 = ["가", "나", "다", "라"]
for 변수 in 리스트[: :2]:
  print(변수)
```

150

리스트에는 네 개의 문자열이 바인딩돼 있다.

리스트 = ["가", "나", "다", "라"]

for문을 사용해서 다음과 같이 출력하라.

라
다
나
가

정답확인

증감폭을 음수로 설정하면 끝에서 앞방향으로 값을 슬라이싱합니다.

```
리스트 = ["가", "나", "다", "라"]
for 변수 in 리스트[: :-1]:
  print(변수)
```

151

리스트에는 네 개의 정수가 저장돼 있다.

리스트 = [3, -20, -3, 44]

for문을 사용해서 리스트의 음수를 출력하라.

-20
-3

정답확인

리스트에 저장된 모든 값에 접근해서 음수 여부를 판단해야 합니다. 라인 3을 제외하면 리스트에 저장된 모든 값을 출력하는 코드입니다. 라인 3의 분기문의 사용으로 모든 값을 출력하는 것이 아니라 조건이 참인 경우에만 if문의 들여쓰기된 코드가 실행됩니다.

```
1: 리스트 = [3, -20, -3, 44]
2: for 변수 in 리스트:
3:    if 변수 < 0:
4:       print(변수)
```

인터프리터가 실행하는 순서를 정리해보면 다음과 같습니다.

(변수 3 바인딩 - 분기문 비교:거짓)
(변수 -20 바인딩 - 분기문 비교:참 - 코드 실행)

(변수 -3 바인딩 - 분기문 비교:참 - 코드 실행)
(변수 44 바인딩 - 분기문 비교:거짓)

152
for문을 사용해서 3의 배수만을 출력하라.

리스트 = [3, 100, 23, 44]
3
정답확인
3의 배수일 경우에만 값을 출력합니다. 3의 배수는 3으로 나눈 나머지가 0이면 3의 배수입니다. 변수 % 3은 변수를 3으로 나눈 나머지를 의미합니다. 예를들어 10 % 3은 10을 3으로 나눈 나머지로 1을 반환합니다.

```
리스트 = [3, 100, 23, 44]
for 변수 in 리스트:
  if 변수 % 3 == 0:
    print(변수)
```

153
리스트에서 20 보다 작은 3의 배수를 출력하라

리스트 = [13, 21, 12, 14, 30, 18]
12
18
정답확인
하나 이상의 조건을 비교할 때는 논리 연산자를 사용해야겠죠? 두 조건이 모두 참일 때만 실행돼야 하므로 and 연산자를 사용합니다. 또한 코드의 가독성 향상을 위해 두 조건을 괄호로 구분했습니다. 괄호가 없어도 코드는 같은 동작을 합니다.

```
리스트 = [13, 21, 12, 14, 30, 18]
for 변수 in 리스트:
  if (변수 < 20) and (변수 % 3 == 0):
    print(변수)
```

and, or, not이 기억나지 않으신 분들은 3장 분기문을 복습해주세요.

154
리스트에서 세 글자 이상의 문자를 화면에 출력하라

리스트 = ["I", "study", "python", "language", "!"]
study
python
language
정답확인

모든 문자열 중에서 문자열의 길이가 3이상인 문자열만 출력합니다. 문자열의 길이를 계산하는 len() 함수와 분기문을 사용합니다.

```
리스트 = ["I", "study", "python", "language", "!"]
for 변수 in 리스트:
  if len(변수) >= 3:
    print(변수)
```

155

리스트에서 대문자만 화면에 출력하라.

```
리스트 = ["A", "b", "c", "D"]
```

A

D

(참고) isupper() 메서드는 대문자 여부를 판별합니다.

```
>> 변수 = "A"
>> 변수.isupper()
True
>> 변수 = "a"
>> 변수.isupper()
False
```

정답확인

```
리스트 = ["A", "b", "c", "D"]
for 변수 in 리스트:
  if 변수.isupper():
    print(변수)
```

156

리스트에서 소문자만 화면에 출력하라.

```
리스트 = ["A", "b", "c", "D"]
```

b

c

정답확인

비교 연산자를 사용해서 다음과 같이 표현할 수 있습니다.

```
리스트 = ["A", "b", "c", "D"]
for 변수 in 리스트:
  if 변수.isupper() == False:
    print(변수)
리스트 = ["A", "b", "c", "D"]
for 변수 in 리스트:
```

```
  if 변수.isupper() != True:
    print(변수)
```
논리 연산자 not을 사용할 수도 있습니다.

```
리스트 = ["A", "b", "c", "D"]
for 변수 in 리스트:
  if not 변수.isupper():
    print(변수)
```

157
이름의 첫 글자를 대문자로 변경해서 출력하라.

```
리스트 = ['dog', 'cat', 'parrot']
Dog
Cat
Parrot
```
(참고) upper() 메서드는 문자열을 대문자로 변경합니다.

```
>> 변수 = "a"
>> a.upper()
A
>> 변수 = "abc"
>> 변수.upper()
ABC
```
정답확인
지금까지 배웠던 내용을 모두 응용해야 하는 문제입니다. 첫 번째 단어만 대문자로 변경해야 하기 때문에 아래의 순서로 처리해야 합니다.
1) 인덱싱으로 첫번째 문자를 가져온다.
2) upper() 함수로 대문자로 변경한다.
3) 변경한 대문자와 나머지 문자를 이어붙인다.
정리한 내용을 코드로 작성하면 다음과 같습니다.
```
리스트 = ['dog', 'cat', 'parrot']
for 변수 in 리스트:
  첫글자 = 변수[0]                # 1)
  대문자 = 첫글자.upper()       # 2)
  print(대문자 + 변수[1:])        # 3)
```
간단하기 때문에 한줄에 코드를 작성해도 좋습니다.

```
for 변수 in 리스트:
  print(변수[0].upper() + 변수[1:])
```

158
파일 이름이 저장된 리스트에서 확장자를 제거하고 파일 이름만 화면에 출력하라. (힌트: split() 메서드)

```
리스트 = ['hello.py', 'ex01.py', 'intro.hwp']
hello
ex01
intro
```

정답확인
split() 메서드는 문자열을 입력된 구분자로 분할해서 리스트로 반환합니다. 다음 예제 코드의 결과를 예상해봅시다.

```
변수 = "abcdef"
print(변수.split("c"))
```

입력된 구분자 ("c")로 문자열을 분할해서 "ab"와 "def"를 리스트에 저장합니다.

```
['ab', 'def']
```

split() 메서드를 사용해서 마침표를 구분자 (".")로 문자열을 분할합니다. 그리고 0번 인덱스에 들어있는 파일 이름만 화면에 출력합니다.

```
리스트 = ['hello.py', 'ex01.py', 'intro.hwp']
for 변수 in 리스트:
  split = 변수.split(".")
  print(split[0])
```

159
파일 이름이 저장된 리스트에서 확장자가 .h인 파일 이름을 출력하라.

```
리스트 = ['intra.h', 'intra.c', 'define.h', 'run.py']
intra.h
define.h
```

정답확인
split() 메서드로 문자열을 분할하고 확장자가 "h"인지 분기문으로 비교합니다. 확장자가 "h"라면 파일 이름을 출력합니다.

```
리스트 = ['intra.h', 'intra.c', 'define.h', 'run.py']
for 변수 in 리스트:
  split = 변수.split(".")
  if split[1] == "h":
    print(변수)
```

160
파일 이름이 저장된 리스트에서 확장자가 .h나 .c인 파일을 화면에 출력하라.

```
리스트 = ['intra.h', 'intra.c', 'define.h', 'run.py']
intra.h
intra.c
define.h
```
정답확인

논리 연산자 or을 사용해서 두 개의 확장자를 비교합니다.

```
리스트 = ['intra.h', 'intra.c', 'define.h', 'run.py']
for 변수 in 리스트:
  split = 변수.split(".")
  if (split[1] == "h") or (split[1] == "c"):
    print(변수)
```

161

for문과 range 구문을 사용해서 0~99까지 한 라인에 하나씩 순차적으로 출력하는 프로그램을 작성하라.

정답확인

아래처럼 print문을 100번 칠 수는 없습니다.

```
print(0)
print(1)
....
print(99)
```
range문은 범위를 만들어 줍니다. 주의 할 점은 0 부터 2 까지 세 개의 데이터를 만들어줍니다.

```
>> list(range (3))
[0, 1, 2]
```
range(100)은 0 부터 99까지 숫자 리스트를 만듭니다. 이를 for문과 함께 사용하면 range로 생성한 모든 숫자르 하나씩 화면에 출력합니다.

```
for i in range(100):
    print(i)
```

162

월드컵은 4년에 한 번 개최된다. range()를 사용하여 2002~2050년까지 중 월드컵이 개최되는 연도를 출력하라.

```
2002
2006
2010
```

...
2042
2046
2050
참고) range의 세번 째 파라미터는 증감폭을 결정합니다.

```
>> print(list(range(0, 10, 2)))
[0, 2, 4, 6, 8]
```

정답확인

```
for x in range(2002, 2051, 4) :
    print (x)
```

163

1부터 30까지의 숫자 중 3의 배수를 출력하라.

3
6
9
12
15
18
21
24
27
30

정답확인

```
for num in range(3, 31, 3):
    print (num)
```

164

99부터 0까지 1씩 감소하는 숫자들을, 한 라인에 하나씩 출력하라.

정답확인

빨셈을 사용해서 원하는 결과를 얻어냅니다.

```
for i in range(100):
    print(99 - i)
```

165

for문을 사용해서 아래와 같이 출력하라.

0.0

0.1
0.2
0.3
0.4
0.5
...
0.9
정답확인
for num in range(10) :
    print(num / 10)

166
구구단 3단을 출력하라.

3x1 = 3
3x2 = 6
3x3 = 9
3x4 = 12
3x5 = 15
3x6 = 18
3x7 = 21
3x8 = 24
3x9 = 27
정답확인
for i in range(1, 10) :
    print (3, "x", i, " = ", 3 * I)

167
구구단 3단을 출력하라. 단 홀수 번째만 출력한다.

3x1 = 3
3x3 = 9
3x5 = 15
3x7 = 21
3x9 = 27
정답확인
range(1, 10, 2)를 사용해서 홀수를 만듭니다.
num = 3
for i in range(1, 10, 2) :
    print (num, "x", i, " = ", num * i)
혹은 조건문을 사용해서 해결할 수도 있습니다.

```
num = 3
for i in range(1, 10) :
    if i % 2 == 1 :
        print (num, "x", i, " = ", num * I)
```

168
1~10까지의 숫자에 대해 모두 더한 값을 출력하는 프로그램을 for 문을 사용하여 작성하라.

합 : 55
정답확인
hab 이라는 변수에 0을 저장하고, for 문을 통해 모든 값에 대해 누적합니다.
```
hab = 0
for i in range(1, 11):
    hab += i
print ("합 :", hab)
```
hab += i 는 아래 코드를 축약해서 작성한 것입니다.

```
hab = hab + I
```

169
1~10까지의 숫자 중 모든 홀수의 합을 출력하는 프로그램을 for 문을 사용하여 작성하라.

합: 25
정답확인
hab 이라는 변수에 0을 저장하고, for 문을 통해 모든 값에 대해 누적합니다. 이때 range의 세 번째 파라미터를 사용해서 range가 생성하는 수를 2씩 증가시킵니다.
```
hab = 0
for i in range(1, 11, 2):
    hab += i
print ("합 :", hab)
```

170
1~10까지의 숫자를 모두 곱한 값을 출력하는 프로그램을 for 문을 사용하여 작성하라.

정답확인
```
result = 1
for i in range(1, 11) :
    result *= i
print(result)
```
result *= i 는 아래 코드를 축약해서 작성한 것입니다.

result = result * I

171
아래와 같이 리스트의 데이터를 출력하라. 단, for문과 range문을 사용하라.

price_list = [32100, 32150, 32000, 32500]
32100
32150
32000
32500
정답확인
for i in range(4):
    print(price_list[i])
len() 함수를 사용하면 price_list 가 변해도 코드의 수정이 필요없습니다. 아래가 더 좋은 코드입니다.

for i in range(len(price_list)):
    print(price_list[i])

172
아래와 같이 리스트의 데이터를 출력하라. 단, for문과 range문을 사용하라.

price_list = [32100, 32150, 32000, 32500]
0 32100
1 32150
2 32000
3 32500
정답확인
price_list = [32100, 32150, 32000, 32500]
for i, data in enumerate(price_list):
    print(i, data)

173
아래와 같이 리스트의 데이터를 출력하라. 단, for문과 range문을 사용하라.

price_list = [32100, 32150, 32000, 32500]
3 32100
2 32150
1 32000
0 32500

정답확인

```
for i in range(len(price_list)):
    print(3 - i, price_list[i])
```

3이라는 숫자보다는 일반적인 형태로 아래와 같이 코딩하는게 좋습니다.

```
for i in range(len(price_list)):
    print((len(price_list) - 1) - i, price_list[i])
```

174

아래와 같이 리스트의 데이터를 출력하라. 단, for문과 range문을 사용하라.

```
price_list = [32100, 32150, 32000, 32500]
```

100 32150

110 32000

120 32500

정답확인

```
for i in range(1, 4):
    print(90 + 10 * i, price_list[i])
```

175

my_list를 아래와 같이 출력하라.

```
my_list = ["가", "나", "다", "라"]
```

가 나

나 다

다 라

정답확인

어렵다면 한단계씩 생각해 봅시다. for문을 사용하지 않고 인덱싱만을 사용해서 코드를 작성해보면 인덱스간의 규칙관계가 눈에 들어옵니다. 같은 행에 있는 두 개의 데이터는 인덱스가 +1 차이납니다. 또한 행이 증가할 때마다 인덱스가 +1 씩 증가합니다.

```
print(my_list[0], my_list[1])
print(my_list[1], my_list[2])
print(my_list[2], my_list[3])
```

분석한 규칙을 바탕으로 반복문을 작성합시다. 아래는 첫 열 "가 나 다" 를 한 라인에 하나씩 출력하는 코드입니다. for 문이 인덱스를 0, 1, 2 차례로 바인딩하고 인덱싱을 사용해 값을 출력합니다.

```
for i in [0, 1, 2]:
    print(my_list[i])
```

한 행에 두 개의 데이터를 출력하고 싶기 때문에 이전 코드의 print 문에 출력하고 싶은 데이터를 추가합니다. 같은 행의 두 데이터는 인덱스 차이가 +1 이라는 것을 잊지마세요. i가 0일

때는 0, 1 위치의 값이 출력됩니다. i가 1일 때는 1, 2 위치의 값이 출력됩니다. * i가 2일 때는 2, 3 위치의 값이 출력됩니다.

```
for i in [0, 1, 2]:
    print(my_list[i], my_list[i+1])
```

위의 코드를 사용해도 원하는 출력값을 얻을 수 있지만 아래는 코드를 보다 일반적인 형태로 변경했습니다. len 함수를 사용한 것을 눈여겨 보세요.

```
for i in range( len(my_list) - 1 ) :
  print(my_list[i], my_list[i+1])
```

아래와 같이 작성해도 됩니다. 인덱스를 갖고 노는 겁니다. for문을 단계별로 풀어 써가며 확인해보세요.

```
for i in range( 1, len(my_list) ) :
  print(my_list[i-1], my_list[i])
```

176

리스트를 아래와 같이 출력하라.

```
my_list = ["가", "나", "다", "라", "마"]
```

가 나 다
나 다 라
다 라 마
정답확인

인덱싱을 사용해서 코드를 작성해보면 인덱스간의 규칙관계가 눈에 들어옵니다. 같은 행에 있는 두 개의 데이터는 인덱스가 +1 차이납니다. 또한 행이 증가할 때마다 인덱스가 +1 씩 증가합니다. 045번 문제와 다른점은 한 행에 출력되는 데이터가 3개라는 것입니다.

```
print(my_list[0], my_list[1], my_list[2])
print(my_list[1], my_list[2], my_list[3])
print(my_list[2], my_list[3], my_list[4])
```

for문내의 print 문에 출력하고 싶은 값의 위치를 나열합니다. i가 0일 때는 0, 1, 2 위치의 값이 출력됩니다. i가 1일 때는 1, 2, 3 위치의 값이 출력됩니다. * i가 2일 때는 2, 3, 4 위치의 값이 출력됩니다.

```
for i in [0, 1, 2]:
    print(my_list[i], my_list[i+1], my_list[i+2])
```

보다 일반적인 형태로 코드를 변경해 봅시다. 아래 세 개의 코드는 모두 동일한 결과를 출력합니다. 인덱스를 만들어 내는 것과, 인덱싱의 코드가 변하는 것을 눈여겨 보세요.

```
for i in range( len(my_list) - 2 ):
    print(my_list[i], my_list[i+1], my_list[i+2])
```

```
for i in range( 1, len(my_list) - 1 ):
    print(my_list[i-1], my_list[i], my_list[i+1])
for i in range( 2, len(my_list) ):
    print(my_list[i-2], my_list[i-1], my_list[i])
```

177
반복문과 range 함수를 사용해서 my_list를 아래와 같이 출력하라.

```
my_list = ["가", "나", "다", "라"]
```
라 다
다 나
나 가
정답확인
어렵다면 우선은 for문을 사용하지 않고 아래와 같이 인덱싱을 사용해서 코드를 작성합니다. 규칙이 눈에 들어오나요?
```
print(my_list[3], my_list[2])
print(my_list[2], my_list[1])
print(my_list[1], my_list[0])
```
각 라인마다 인덱스가 1씩 감소하고, 같은 라인에서 우측 인덱스는 좌측 보다 1 작다고 분석할 수 있습니다. 이를 코드로 옮기면 아래와 같습니다.

```
for i in [3, 2, 1] :
    print(my_list[i], my_list[i-1])
```
인덱스를 만들어내는 방법에 따라 호출하는 코드가 달라집니다. 우변을 기준으로 인덱스를 만들고 좌변을 접근할 때는 +1 이 필요합니다.

```
for i in [2, 1, 0] :
    print(my_list[i+1], my_list[i])
for i in [0, 1, 2] :
    print(my_list[3 - i], my_list[2 - i])
```
일반적인 형태로 코드를 수정하면 아래와 같습니다.

```
for i in range(len(my_list) - 1, 0, -1):
    print(my_list[i], my_list[i-1])
```
인덱스를 만들어내는 방법에 따라 답은 여러개가 될 수 있습니다.

```
for i in range(len(my_list) - 1):
    print(my_list[len(my_list) - 1 - i], my_list[len(my_list) - 2 - i])
```

178
리스트에는 네 개의 정수가 저장되어 있다. 각각의 데이터에 대해서 자신과 우측값과의 차분

값을 화면에 출력하라.

my_list = [100, 200, 400, 800]
예를들어 100을 기준으로 우측에 위치한 200과의 차분 값를 화면에 출력하고, 200을 기준으로 우측에 위치한 400과의 차분값을 화면에 출력한다. 이어서 400을 기준으로 우측에 위치한 800과의 차분값을 화면에 출력한다.

100
200
400
정답확인
한 번에 코드를 작성하기 어렵다면 print 함수와 인덱싱만으로 결과를 표현해봅시다.
print(abs(my_list[1] - my_list[0]))
print(abs(my_list[2] - my_list[1]))
print(abs(my_list[3] - my_list[2]))
인덱스의 관계를 파악했다면 기준으로 사용할 인덱스를 결정합니다. 이번에는 우측에 있는 인덱스를 활용해서 for문을 작성해 보겠습니다.

```
for i in [0, 1, 2]:
    print(abs(my_list[i+1] - my_list[i]))
```
다음으로 코드를 다듬어 줍니다.

```
for i in range(len(my_list) - 1):
    print(abs(my_list[i+1] - my_list[i]))
```

179
리스트에는 6일 간의 종가 데이터가 저장되어 있다. 종가 데이터의 3일 이동 평균을 계산하고 이를 화면에 출력하라.

my_list = [100, 200, 400, 800, 1000, 1300]
첫 번째 줄에는 100, 200, 400의 평균값이 출력된다. 두 번째 줄에는 200, 400, 800의 평균값이 출력된다. 같은 방식으로 나머지 데이터의 평균을 출력한다.

233.33333333333334
466.6666666666667
733.3333333333334
1033.3333333333333
정답확인
print 함수와 인덱싱만으로 결과를 표현해봅니다.
print(abs(my_list[0] + my_list[1] + my_list[2])/3)
print(abs(my_list[1] + my_list[2] + my_list[3])/3)

```
print(abs(my_list[2] + my_list[3] + my_list[4])/3)
print(abs(my_list[3] + my_list[4] + my_list[5])/3)
```

인덱스 간의 관계를 파악했다면 기준으로 사용할 인덱스를 결정합니다. 이번에는 가운데에 있는 인덱스를 활용해서 for문을 작성해 보겠습니다.

```
for i in [1, 2, 3, 4]:
    print(abs(my_list[i-1] + my_list[i] + my_list[i+1]) / 3)
```

다음으로 코드를 다듬어 줍니다.

```
for i in range(1, len(my_list) - 1):
    print(abs(my_list[i-1] + my_list[i] + my_list[i+1]) / 3)
```

180

리스트에 5일간의 저가, 고가 정보가 저장돼 있다. 고가와 저가의 차를 변동폭이라고 정의할 때, low, high 두 개의 리스트를 사용해서 5일간의 변동폭을 volatility 리스트에 저장하라.

```
low_prices  = [100, 200, 400, 800, 1000]
high_prices = [150, 300, 430, 880, 1000]
```

정답확인

```
volatility = []
for i in range(len(low_prices)) :
    volatility.append(high_prices[i] - low_prices[i])
```

181

아래 표에서 하나의 행을 하나의 리스트로, 총 3개의 리스트를 갖는 이차원 리스트 apart를 정의하라.

101호	102호
201호	202호
301호	302호

정답확인

```
apart = [ ["101호", "102호"], ["201호", "202호"], ["301호", "302호"] ]
```

182

아래 표에서 하나의 열을 하나의 리스트로, 총 2개의 리스트를 갖는 이차원 리스트 stock을 정의하라.

시가	종가
100	80
200	210
300	330

정답확인
stock = [["시가", 100, 200, 300], ["종가", 80, 210, 330]]

183
아래 표를 stock 이름의 딕셔너리로 표현하라.시가를 key로 저장하고, 나머지 같은 열의 데이터를 리스트로 저장해서 value로 저장한다. 종가 역시 key로 저장하고 나머지 같은 열의 데이터를 리스트로 저장해서 value로 저장한다.

시가	종가
100	80
200	210
300	330

정답확인
stock = {"시가": [100, 200, 300], "종가": [80, 210, 330] }

184
아래 표를 stock 이라는 이름의 딕셔너리로 표현하라. 날짜를 key로 저장하고, 나머지 같은 행의 데이터를 리스트로 저장해서 value로 저장한다. 첫 열이 날짜이다.

10/10	80	110	70	90
10/11	210	230	190	200

정답확인
stock = {"10/10": [80, 110, 70, 90], "10/11": [210, 230, 190, 200] }

185
리스트에 저장된 데이터를 아래와 같이 출력하라.

apart = [[101, 102], [201, 202], [301, 302]]
101 호
102 호
201 호
202 호
301 호
302 호
정답확인
```
for row in apart:
    for col in row:
        print(col, "호")
```

186
리스트에 저장된 데이터를 아래와 같이 출력하라.

```
apart = [ [101, 102], [201, 202], [301, 302] ]
301 호
302 호
201 호
202 호
101 호
102 호
정답확인
for row in apart[::-1]:
    for col in row:
        print(col, "호")
```

187

리스트에 저장된 데이터를 아래와 같이 출력하라.

```
apart = [ [101, 102], [201, 202], [301, 302] ]
302 호
301 호
202 호
201 호
102 호
101 호
정답확인
for row in apart[::-1]:
    for col in row[::-1]:
        print(col, "호")
```

188

리스트에 저장된 데이터를 아래와 같이 출력하라.

```
apart = [ [101, 102], [201, 202], [301, 302] ]
101 호
-----
102 호
-----
201 호
-----
202 호
-----
301 호
```

```
-----
302 호
-----
```
정답확인

```
for row in apart:
    for col in row:
        print(col, "호")
        print("-" * 5)
```

189

리스트에 저장된 데이터를 아래와 같이 출력하라.

```
apart = [ [101, 102], [201, 202], [301, 302] ]
101 호
102 호
-----
201 호
202 호
-----
301 호
302 호
-----
```
정답확인

```
for row in apart:
    for col in row:
        print(col, "호")
    print("-----")
```

190

리스트에 저장된 데이터를 아래와 같이 출력하라.

```
apart = [ [101, 102], [201, 202], [301, 302] ]
101 호
102 호
201 호
202 호
301 호
302 호
-----
```
정답확인

들여쓰기가 변경됐습니다. 눈여겨보세요.

```
for row in apart:
    for col in row:
        print(col, "호")
print("-" * 5)
```

191

data에는 매수한 종목들의 OHLC (open/high/low/close) 가격 정보가 바인딩 되어있다.

```
data = [
    [ 2000,  3050,  2050,  1980],
    [ 7500,  2050,  2050,  1980],
    [15450, 15050, 15550, 14900]
]
```

수수료를 0.014 %로 가정할 때, 각 가격에 수수료를 포함한 가격을 한라인에 하나씩 출력하라.

2000.28
3050.427
2050.2870000000003
...
정답확인

```
for line in data:
    for column in line:
        print(column * 1.00014)
```

192

191번의 출력 결과에 행단위로 "----" 구분자를 추가하라.

2000.28
3050.427
2050.2870000000003
1980.2772

7501.05
2050.2870000000003
2050.2870000000003
1980.2772

15452.163
15052.107

15552.177
14902.086000000001

정답확인
```
for line in data:
    for column in line:
        print(column * 1.00014)
    print("----")
```

193
192 번 문제의 결괏값을 result 이름의 리스트에 1차원 배열로 저장하라.

```
>> print(result)
[2000.28, 3050.427, 2050.2870000000003, 1980.2772, 7501.05, 2050.2870000000003, 2050.2870000000003, ...]
```
정답확인
```
result = []
for line in data:
    for column in line:
        result.append(column * 1.00014)
print(result)
```

194
191번 문제의 결괏값을 result 이름의 리스트에 2차원 배열로 저장하라. 저장 포맷은 아래와 같다. 각 행에 대한 데이터끼리 리스트에 저장되어야 한다.

```
>> print(result)
[
 [2000.28, 3050.427, 2050.2870000000003, 1980.2772],
 [7501.05, 2050.2870000000003, 2050.2870000000003, 1980.2772],
 [15452.163, 15052.107, 15552.177, 14902.086000000001]
]
```
정답확인
```
result = []
for line in data:
    sub = []
    for column in line:
        sub.append(column * 1.00014)
    result.append(sub)
print(result)
```

195
ohlc 리스트에는 시가(open), 고가 (high), 저가 (low) , 종가(close)가 날짜별로 저장돼 있다. 화면에 종가데이터를 출력하라.

```
ohlc = [["open", "high", "low", "close"],
        [100, 110, 70, 100],
        [200, 210, 180, 190],
        [300, 310, 300, 310]]
```

100
190
310
정답확인

```
for row in ohlc[1:]:
    print(row[3])
```

196
ohlc 리스트에는 시가(open), 고가 (high), 저가 (low) , 종가(close)가 날짜별로 저장돼 있다. 종가가 150원보다 큰경우에만 종가를 출력하라.

```
ohlc = [["open", "high", "low", "close"],
        [100, 110, 70, 100],
        [200, 210, 180, 190],
        [300, 310, 300, 310]]
```

190
310
정답확인

```
for row in ohlc[1:]:
    if (row[3] > 150)
        print(row[3])
```

197
ohlc 리스트에는 시가(open), 고가 (high), 저가 (low) , 종가(close)가 날짜별로 저장돼 있다. 종가가 시가 보다 크거나 같은 경우에만 종가를 출력하라.

```
ohlc = [["open", "high", "low", "close"],
        [100, 110, 70, 100],
        [200, 210, 180, 190],
        [300, 310, 300, 310]]
```

100
310

정답확인

```
for row in ohlc[1:]:
    if ((row[3] == row[0]) or (row[3] > row[0])):
        print(row[3])
```

198

ohlc 리스트에는 시가(open), 고가 (high), 저가 (low) , 종가(close)가 날짜별로 저장돼 있다. 고가와 저가의 차이를 변동폭으로 정의할 때 변동폭을 volatility 이름의 리스트에 저장하라.

```
ohlc = [["open", "high", "low", "close"],
        [100, 110, 70, 100],
        [200, 210, 180, 190],
        [300, 310, 300, 310]]
>> print(volatility)
[40, 30, 10]
```

정답확인

```
volatility = []
for row in ohlc[1:]:
    volatility.append(row[1]-row[2])
print(volatility)
```

range 구문을 사용해서도 문제를 풀 수 있습니다.

```
volatility = []
for i in range(1, len(ohlc)):
    volatility.append(ohlc[i][1]-ohlc[i][2])
print(volatility)
```

199

리스트에는 3일 간의 ohlc 데이터가 저장돼 있다. 종가가 시가보다 높은 날의 변동성 (고가 - 저가)을 화면에 출력하라.

```
ohlc = [["open", "high", "low", "close"],
        [100, 110, 70, 100],
        [200, 210, 180, 190],
        [300, 310, 300, 310]]
```

종가가 시가보다 높은 거래일의 OHLC는 [300, 310, 300, 310] 이다. 따라서 이 거래일의 변동성은 10 (310 - 300)이다.

10

정답확인

```
for row in ohlc[1:]:
    if row[3] > row[0]:
        print(row[1]-row[2])
```

200

리스트에는 3일 간의 ohlc 데이터가 저장돼 있다. 시가에 매수해서 종가에 매도 했을 경우 총 수익금을 계산하라.

```
ohlc = [["open", "high", "low", "close"],
        [100, 110, 70, 100],
        [200, 210, 180, 190],
        [300, 310, 300, 310]]
```

1일차 수익 0원 (100 - 100), 2일차 수익 -10원 (190 - 200), 3일차 수익 10원 (310 - 300)이다.

0

정답확인

```
profit = 0
for row in ohlc[1:]:
    profit += (row[3] - row[0])
```

201

"비트코인" 문자열을 화면에 출력하는 print_coin() 함수를 정의하라.

정답확인

```
def print_coin():
    print("비트코인")
```

202

201번에서 정의한 함수를 호출하라.

정답확인

```
print_coin()
```

203

201번에서 정의한 print_coin 함수를 100번호출하라.

정답확인

```
for i in range(100):
    print_coin()
```

204
"비트코인" 문자열을 100번 화면에 출력하는 print_coins() 함수를 정의하라.

정답확인
```
def print_coins():
    for i in range(100):
        print("비트코인")
```

205
아래의 에러가 발생하는 이유에 대해 설명하라.

```
hello()
def hello():
    print("Hi")
```
실행 예

```
NameError: name 'hello' is not defined
```
정답확인
함수가 정의되기 전에 호출되어서 에러가 발생합니다.

206
아래 코드의 실행 결과를 예측하라.

```
def message() :
    print("A")
    print("B")

message()
print("C")
message()
```
정답확인
```
A
B
C
A
B
```

207
아래 코드의 실행 결과를 예측하라. (읽기 어려운 코드의 예입니다.)

```
print("A")

def message() :
    print("B")

print("C")
message()
```

정답확인

A
C
B

208

아래 코드의 실행 결과를 예측하라. (읽기 어려운 코드의 예입니다.)

```
print("A")
def message1() :
    print("B")
print("C")
def message2() :
    print("D")
message1()
print("E")
message2()
```

정답확인

A
C
B
E
D

209

아래 코드의 실행 결과를 예측하라.

```
def message1():
    print("A")

def message2():
    print("B")
    message1()
```

message2()
정답확인
B
A

210
아래 코드의 실행 결과를 예측하라.

```
def message1():
    print("A")

def message2():
    print("B")

def message3():
    for i in range (3) :
        message2()
        print("C")
    message1()

message3()
```

정답확인
B
C
B
C
B
C
A
==========
211
함수의 호출 결과를 예측하라.

```
def 함수(문자열) :
    print(문자열)

함수("안녕")
함수("Hi")
```

정답확인
안녕
Hi

212
함수의 호출 결과를 예측하라.

```
def 함수(a, b) :
    print(a + b)

함수(3, 4)
함수(7, 8)
```

정답확인
7
15

213
아래와 같은 에러가 발생하는 원인을 설명하라.

```
def 함수(문자열) :
    print(문자열)
함수()
```

TypeError: 함수() missing 1 required positional argument: '문자열'
정답확인
함수에 정의와 다르게 함수를 호출하고 있다. 함수를 호출할 때 하나의 파라미터를 입력해야 한다.

214
아래와 같은 에러가 발생하는 원인을 설명하라.

```
def 함수(a, b) :
    print(a + b)

함수("안녕", 3)
```

TypeError: must be str, not int
정답확인
정의된 함수는 같은 타입의 두 개의 값을 입력 받아 덧셈 연산을 적용하려는 의도로 설계됐습니다. 하지만 함수를 호출 할때 문자열과 숫자를 입력해서 문자열과 숫자는 더할 수 없다는 에러가 발생합니다.

215
하나의 문자를 입력받아 문자열 끝에 ":D" 스마일 문자열을 이어 붙여 출력하는 print_with_smile 함수를 정의하라.

정답확인
```
def print_with_smile (string) :
    print (string + ":D")
```

216
215에서 정의한 함수를 호출하라. 파라미터는 "안녕하세요"로 입력하라.

정답확인
```
print_with_smile("안녕하세요")
```

217
현재 가격을 입력 받아 상한가 (30%)를 출력하는 print_upper_price 함수를 정의하라.

정답확인
```
def print_upper_price(price) :
    print(price * 1.3)
```

218
두 개의 숫자를 입력받아 두 수의 합을 출력하는 print_sum 함수를 정의하라.

정답확인
```
def print_sum (a, b) :
    print (a + b)
```

219
두 개의 숫자를 입력받아 합/차/곱/나눗셈을 출력하는 print_arithmetic_operation 함수를 작성하라.

```
print_arithmetic_operation(3, 4)
3 + 4 = 7
3 - 4 = -1
3 * 4 = 12
3 / 4 = 0.75
```
정답확인
```
def print_arithmetic_operation(a, b):
    print(a, "+", b, "=", a + b)
    print(a, "-", b, "=", a - b)
    print(a, "*", b, "=", a * b)
    print(a, "/", b, "=", a / b)
```

220

세 개의 숫자를 입력받아 가장 큰수를 출력하는 print_max 함수를 정의하라. 단 if 문을 사용해서 수를 비교하라.

정답확인

```
def print_max(a, b, c) :
    max_val = 0
    if a > max_val :
        max_val = a
    if b > max_val :
        max_val = b
    if c > max_val :
        max_val = c
    print(max_val)
```

지금까지써왔던 max 함수의 내부는 위와 비슷하게 구현되어 있을 겁니다.

```
max(a, b, c)
```

221

입력된 문자열을 역순으로 출력하는 print_reverse 함수를 정의하라.

```
print_reverse("python")
nohtyp
```

정답확인

슬라이싱을 사용하여 문자열을 역순으로 얻어올 수 있습니다.

```
def print_reverse(string) :
    print(string[::-1])
```

222

성적 리스트를 입력 받아 평균을 출력하는 print_score 함수를 정의하라.

```
print_score ([1, 2, 3])
2.0
```

정답확인

```
def print_score(score_list) :
    print(sum(score_list)/len(score_list))
```

223

하나의 리스트를 입력받아 짝수만 화면에 출력하는 print_even 함수를 정의하라.

```
print_even ([1, 3, 2, 10, 12, 11, 15])
2
```

10
12
정답확인
```
def print_even (my_list) :
    for v in my_list :
        if v % 2 == 0 :
            print(v)
```

224
하나의 딕셔너리를 입력받아 딕셔너리의 key 값을 화면에 출력하는 print_keys 함수를 정의하라.

```
print_keys ({"이름":"김말똥", "나이":30, "성별":0})
```
이름
나이
성별
정답확인
```
def print_keys(dic):
    for keys in dic.keys():
        print(keys)
```

225
my_dict에는 날짜를 키값으로 OHLC가 리스트로 저장돼 있다.

```
my_dict = {"10/26" : [100, 130, 100, 100],
           "10/27" : [10, 12, 10, 11]}
```
my_dict와 날짜 키값을 입력받아 OHLC 리스트를 출력하는 print_value_by_key 함수를 정의하라.

```
print_value_by_key  (my_dict, "10/26")
```
[100, 130, 100, 100]
정답확인
```
def print_value_by_key (my_dict, key) :
    print(my_dict[key])
```

226
입력 문자열을 한 줄에 다섯글자씩 출력하는 print_5xn(string) 함수를 작성하라.

```
print_5xn("아이엠어보이유알어걸")
```
아이엠어보
이유알어걸

정답확인

```
def print_5xn(line):
    chunk_num = int(len(line) / 5)
    for x in range(chunk_num + 1) :
        print(line[x * 5: x * 5 + 5])
```

227

문자열과 한줄에 출력될 글자 수를 입력을 받아 한 줄에 입력된 글자 수만큼 출력하는 print_mxn(string) 함수를 작성하라.

printmxn("아이엠어보이유알어걸", 3)

아이엠

어보이

유알어

걸

정답확인

```
def print_mxn(line, num):
    chunk_num = int(len(line) / num)
    for x in range(chunk_num + 1) :
        print(line[x * num: x * num + num])
```

228

연봉을 입력받아 월급을 계산하는 calc_monthly_salary(annual_salary) 함수를 정의하라. 회사는 연봉을 12개월로 나누어 분할 지급하며, 이 때 1원 미만은 버림한다.

calc_monthly_salary(12000000)

1000000

정답확인

입력된 값을 12로 나누고 형변환을 해서 1원 미만을 절사합니다.

```
def calc_monthly_salary(annual_pay) :
    monthly_pay = int(annual_pay / 12)
    return monthly_pay
```

229

아래 코드의 실행 결과를 예측하라.

```
def my_print (a, b) :
    print("왼쪽:", a)
    print("오른쪽:", b)

my_print(a=100, b=200)
```

정답확인
우선 아래의 기본 코드를 분석해 보겠습니다. 5번 라인에서 함수가 호출되면, 함수가 정의된 1번 라인으로 이동합니다. 이 때 함수의 입력 변수 a에 100을 바인딩하고, b에 200을 바인딩합니다.
my_print(100, 200)
아래 코드는 함수가 호출될 때 a에 100, b에 200을 바인딩하라는 것을 명시적으로 표현한 코드입니다. 아래 코드는 문법만 다를 뿐 위 코드와 동일한 동작을 수행합니다.

my_print(a=100, b=200)
함수의 호출 결과는 아래와 같습니다.

왼쪽: 100
오른쪽: 200

230
아래 코드의 실행 결과를 예측하라.

```
def my_print (a, b) :
    print("왼쪽:", a)
    print("오른쪽:", b)

my_print(b=100, a=200)
```

정답확인
5 번 라인에서 함수를 호출하는데, a 변수에 200, b 변수에 100을 바인딩하라고 지정을 했습니다. 따라서 아래와 같이 출력됩니다.
왼쪽: 200
오른쪽: 100

231
아래 코드를 실행한 결과를 예상하라.

```
def n_plus_1 (n) :
    result = n + 1

n_plus_1(3)
print (result)
```

정답확인
에러가 발생합니다.

NameError Traceback (most recent call last)
<ipython-input-2-78e20c8ecef0> in <module>()

3
4 n_plus_1(3)
----> 5 print (result)
6

NameError: name 'result' is not defined
함수 내부에서 사용한 변수는 함수 밖에서는 접근이 불가능합니다. (문법이 그래요) 함수 내부에서 계산한 값을 전달하기 위해서는 return을 사용해야 합니다.

232
문자열 하나를 입력받아 인터넷 주소를 반환하는 make_url 함수를 정의하라.

make_url("naver")
www.naver.com
정답확인
```
def make_url(string) :
    url = "www." + string + ".com"
    return url
```
간단하므로 한줄로 표현할 수도 있습니다.

```
def make_url(string) :
    return "www." + string + ".com“
```

233
문자열을 입력받아 각 문자들로 구성된 리스트로 반환하는 make_list 함수를 정의하라.

make_list("abcd")
['a', 'b', 'c', 'd']
정답확인
비어있는 리스트에 문자열을 하나씩 추가합니다. 이어서 리스트를 반환 (return) 합니다.
```
def make_list (string) :
    my_list = []
    for 변수 in string :
        my_list.append(변수)
    return my_list
```
문자열을 list로 형변환하면 쉽게 문제를 해결할 수 있습니다.

```
def make_list (string) :
    return list(string)
```

234

숫자로 구성된 하나의 리스트를 입력받아, 짝수들을 추출하여 리스트로 반환하는 pickup_even 함수를 구현하라.

pickup_even([3, 4, 5, 6, 7, 8])
[4, 6, 8]
정답확인
```
def pickup_even(items):
    result = []
    for item in items:
        if item % 2 == 0:
            result.append(item)
    return result
```

235
콤마가 포함된 문자열 숫자를 입력받아 정수로 변환하는 convert_int 함수를 정의하라.

convert_int("1,234,567")
1234567
정답확인
```
def convert_int (string) :
    return int(string.replace(',', ''))
```

236
아래 코드의 실행 결과를 예측하라.

```
def 함수(num) :
    return num + 4

a = 함수(10)
b = 함수(a)
c = 함수(b)
print(c)
```
정답확인
4번 라인에서 함수로 10이 입력돼서 14가 반환됩니다. a 변수에는 14가 저장됩니다. 5번 라인에서 함수로 14가 입력돼서 18이 반환됩니다. 변수 b에는 18이 바인딩됩니다. 6번 라인에서 함수로 18가 입력돼서 22가 반환됩니다. 변수 c에는 22가 바인딩됩니다.

237
아래 코드의 실행 결과를 예측하라.

```
def 함수(num) :
```

```
    return num + 4

c = 함수(함수(함수(10)))
print(c)
```
정답확인
함수가 여러번 중첩돼 있습니다. 안쪽 부터 차례로 해석하면 됩니다. 함수(10)의 결과 14, 함수(14) 결과 18, 함수(18) 결과 22 가 반환됩니다. 결국 36번과 동일한 코드를 축약해서 작성해 놓은 겁니다.

238
아래 코드의 실행 결과를 예측하라.

```
def 함수1(num) :
    return num + 4

def 함수2(num) :
    return num * 10

a = 함수1(10)
c = 함수2(a)
print(c)
```
정답확인
7번 라인에서 함수1으로 10이 입력돼서 14가 반환됩니다. a 변수에는 14가 저장됩니다. 8번 라인에서 함수2로 a에 저장된 14가 입력돼서 140이 반환됩니다. 변수 c에는 140이 바인딩됩니다.
140

239
아래 코드의 실행 결과를 예측하라.

```
def 함수1(num) :
    return num + 4

def 함수2(num) :
    num = num + 2
    return 함수1(num)

c = 함수2(10)
print(c)
```
정답확인
8번 함수2가 호출됩니다. 4번 라인으로 파이썬 인터프리터는 이동하고 이때 num에는 10이

바인딩됩니다. 5번 라인 코드를 실행하면 num이 12로 업데이트 됩니다. 6번라인의 코드를 실행하려고 하는데, 함수1이 호출됩니다. 1번 라인의 함수 정의부로 이동하며 num 값은 12로 바인딩됩니다. 2번 라인의 코드가 실행돼서 16이 반환됩니다. 함수1의 동작을 끝마치고 함수 2의 6번 라인으로 돌아오고 함수2도 return을 만나면서 16을 반환합니다. 8번 라인으로 돌아와 c 변수에는 16을 바인딩합니다.

240

아래 코드의 실행 결과를 예측하라.

```
def 함수0(num) :
    return num * 2

def 함수1(num) :
    return 함수0(num + 2)

def 함수2(num) :
    num = num + 10
    return 함수1(num)

c = 함수2(2)
print(c)
```

정답확인

28

---

241 현재시간

datetime 모듈을 사용해서 현재 시간을 화면에 출력해보세요.

정답확인

```
import datetime

now = datetime.datetime.now()
print(now)
```

2021-07-04 20:31:04.567927

==========

242 현재시간의 타입

datetime 모듈의 now 함수의 리턴 값의 타입을 화면에 출력해보세요.

정답확인

```
import datetime

now = datetime.datetime.now()
```

```
print(now, type(now))
```

2021-07-04 20:30:11.730963 **<class 'datetime.datetime'>**

---

243 timedelta

datetime 모듈의 timedelta를 사용해서 오늘로부터 5일, 4일, 3일, 2일, 1일 전의 날짜를 화면에 출력해보세요.

정답확인

```
import datetime

now = datetime.datetime.now()

for day in range(5, 0, -1):
    delta = datetime.timedelta(days=day)
    date = now - delta
    print(date)
```

```
2021-06-29 20:32:39.351049
2021-06-30 20:32:39.351049
2021-07-01 20:32:39.351049
2021-07-02 20:32:39.351049
2021-07-03 20:32:39.351049
```

---

244 strftime

현재시간을 얻어온 후 다음과 같은 포맷으로 시간을 출력해보세요. strftime 메서드를 사용하세요.

18:35:01

정답확인

```
import datetime

now = datetime.datetime.now()
print(now.strftime("%H:%M:%S"))
```

```
20:33:46
```

---

245 strptime

datetime.datetime.strptime 메서드를 사용하면 문자열 형식의 시간을 datetime.datetime 타입의 시간 값으로 만들어줍니다. "2020-05-04"의 문자열을 시간 타입으로 변환해보세요.

정답확인

```
import datetime

day = "2020-05-04"
ret = datetime.datetime.strptime(day, "%Y-%m-%d")
print(ret, type(ret))
```

```
2020-05-04 00:00:00 <class 'datetime.datetime'>
```

---

246 sleep 함수

time 모듈, datetime 모듈을 사용해서 1초에 한 번 현재 시간을 출력하는 코드를 작성하세요.

정답확인

```
import time
import datetime

while True:
    now = datetime.datetime.now()
    print(now)
    time.sleep(1)
```

```
2021-07-04 20:36:11.469956
2021-07-04 20:36:12.470972
2021-07-04 20:36:13.475958
2021-07-04 20:36:14.476404
```

---

247 모듈 임포트

모듈을 임포트하는 4가지 방식에 대해 설명해보세요.

정답확인

248 os 모듈

os 모듈의 getcwd 함수를 호출하여 현재 디렉터리의 경로를 화면에 출력해보세요.

정답확인

```
import os
ret = os.getcwd()
print(ret, type(ret))
```

C:\Users\pc\PycharmProjects\openapi <class 'str'>
---
249 rename 함수
바탕화면에 텍스트 파일을 하나 생성한 후 os 모듈의 rename 함수를 호출하여 해당 파일의 이름을 변경해보세요.

정답확인
```
import os
os.rename("C:/Users/hyunh/Desktop/before.txt", "C:/Users/hyunh/Desktop/after.txt")
```
---
250 numpy
numpy 모듈의 arange 함수를 사용해서 0.0 부터 5.0까지 0.1씩 증가하는 값을 화면에 출력해보세요.

정답확인
```
import numpy
for i in numpy.arange(0, 5, 0.1):
    print(i)
```

0.0
0.1
0.2
0.30000000000000004
0.4
0.5
0.6000000000000001
0.7000000000000001
0.8
0.9
1.0
---

https://wikidocs.net/7035

251 클래스, 객체, 인스턴스
클래스, 객체, 인스턴스에 대해 설명해봅시다.

정답확인

252 클래스 정의
비어있는 사람 (Human) 클래스를 "정의" 해보세요.

정답확인

```
class Human:
    pass
```

253 인스턴스 생성

사람 (Human) 클래스의 인스턴스를 "생성" 하고 이를 areum 변수로 바인딩해보세요.

정답확인

```
class Human:
    pass

areum = Human()
```

---

254 클래스 생성자-1

사람 (Human) 클래스에 "응애응애"를 출력하는 생성자를 추가하세요.

```
>>> areum = Human()
응애응애
```

정답확인

```
class Human:
    def __init__(self):
        print("응애응애")

areum = Human()
```

---

255 클래스 생성자-2

사람 (Human) 클래스에 (이름, 나이, 성별)을 받는 생성자를 추가하세요.

```
>>> areum = Human("아름", 25, "여자")
```

정답확인

```
class Human:
    def __init__(self, name, age, sex):
        self.name = name
        self.age = age
        self.sex = sex

areum = Human("아름", 25, "여자")
print(areum.name)
```

아름
==========
256 인스턴스 속성에 접근
255에서 생성한 인스턴스의 이름, 나이, 성별을 출력하세요. 인스턴스 변수에 접근하여 값을 출력하면 됩니다.

이름: 조아름, 나이: 25, 성별: 여자
인스턴스 변수에 접근하여 값을 가져오는 예

```
>>> areum.age
25
```

정답확인

```
class Human:
    def __init__(self, name, age, sex):
        self.name = name
        self.age = age
        self.sex = sex

areum = Human("아름", 25, "여자")
print(areum.age)
print("이름:",areum.name)  ????
print("나이:",areum.age)
print("성별:",areum.sex)
```

---
257 클래스 메소드 - 1
사람 (Human) 클래스에서 이름, 나이, 성별을 출력하는 who() 메소드를 추가하세요.

```
>>> areum.who()
이름: 조아름, 나이: 25, 성별: 여자
```

정답확인

```
class Human:
    def __init__(self, name, age, sex):
        self.name = name
        self.age = age
        self.sex = sex

    def who(self):
        print("이름: {} 나이: {} 성별: {}".format(self.name, self.age, self.sex))

areum = Human("아름", 25, "여자")
areum.who()        # Human.who(areum)
```

==========
259 클래스 소멸자
사람 (human) 클래스에 "나의 죽음을 알리지 말라"를 출력하는 소멸자를 추가하세요.

```
>>> areum = Human("아름", 25, "여자")
>>> del areum
```

나의 죽음을 알리지 말라
정답확인

```
class Human:
    def __init__(self, name, age, sex):
        self.name = name
        self.age = age
        self.sex = sex

    def __del__(self):
        print("나의 죽음을 알리지마라")

    def who(self):
        print("이름: {} 나이: {} 성별: {}".format(self.name, self.age, self.sex))

    def setInfo(self, name, age, sex):
        self.name = name
        self.age = age
        self.sex = sex

areum = Human("아름", 25, "여자")
areum.who()
del(areum)
```

==========
이름: 아름 나이: 25 성별: 여자
나의 죽음을 알리지마라
==========
260 에러의 원인
아래와 같은 에러가 발생한 원인에 대해 설명하세요.

```
class OMG :
    def print() :
        print("Oh my god")

>>> >>> myStock = OMG()
>>> myStock.print()
```

```
TypeError Traceback (most recent call last)
<ipython-input-233-c85c04535b22> in <module>()
----> myStock.print()

TypeError: print() takes 0 positional arguments but 1 was given
```

정답확인

```
class OMG :
    def print() :
        print("Oh my god")


mystock = OMG()
mystock.print()        # OMG.print(mystock)
```

==========

261 Stock 클래스 생성

주식 종목에 대한 정보를 저장하는 Stock 클래스를 정의해보세요. 클래스는 속성과 메서드를 갖고 있지 않습니다.

정답확인

```
class Stock:
    pass
```

---

262 생성자

Stock 클래스의 객체가 생성될 때 종목명과 종목코드를 입력 받을 수 있도록 생성자를 정의해보세요.

```
삼성 = Stock("삼성전자", "005930")
```

정답확인

```
class Stock:
    def __init__(self, name, code):
        self.name = name
        self.code = code

삼성 = Stock("삼성전자", "005930")
print(삼성.name)
print(삼성.code)
```

```
삼성전자
005930
```

==========

263 메서드
객체에 종목명을 입력할 수 있는 set_name 메서드를 추가해보세요.

```
a = Stock(None, None)
a.set_name("삼성전자")
```

정답확인

```
class Stock:
    def __init__(self, name, code):
        self.name = name
        self.code = code

    def set_name(self, name):
        self.name = name

a = Stock(None, None)
a.set_name("삼성전자")  # Stock.set_name(a, "삼성전자")
print(a.name)
print(a.code)
```

삼성전자
None

==========

264 메서드
객체에 종목코드를 입력할 수 있는 set_code 메서드를 추가해보세요.

```
a = Stock(None, None)
a.set_code("005930")
```

정답확인

```
class Stock:
    def __init__(self, name, code):
        self.name = name
        self.code = code

    def set_name(self, name):
        self.name = name

    def set_code(self, code):
        self.code = code

a = Stock(None, None)
a.set_code("005930")
```

```
print(a.code)
print(a.name)
```

005930
None
---
265 메서드
종목명과 종목코드를 리턴하는 get_name, get_code 메서드를 추가하세요. 해당 메서드를 사용하여 종목명과 종목코드를 얻고 이를 출력해보세요.

```
삼성 = Stock("삼성전자", "005930")
```
정답확인
```
class Stock:
    def __init__(self, name, code):
        self.name = name
        self.code = code

    def set_name(self, name):
        self.name = name

    def set_code(self, code):
        self.code = code

    def get_name(self):
        return self.name

    def get_code(self):
        return self.code

삼성 = Stock("삼성전자", "005930")
print(삼성.name)
print(삼성.code)
print(삼성.get_name())
print(삼성.get_code())
```

삼성전자
005930
삼성전자
005930
==========
266 객체의 속성값 업데이트

생성자에서 종목명, 종목코드, PER, PBR, 배당수익률을 입력 받을 수 있도록 생성자를 수정하세요. PER, PBR, 배당수익률은 float 타입입니다.

정답확인

```
class Stock:
    def __init__(self, name, code, per, pbr, 배당수익률):
        self.name = name
        self.code = code
        self.per = per
        self.pbr = pbr
        self.배당수익률 = 배당수익률

    def set_name(self, name):
        self.name = name

    def set_code(self, code):
        self.code = code

    def get_name(self):
        return self.name

    def get_code(self):
        return self.code
```

---

267 객체 생성

266번에서 정의한 생성자를 통해 다음 정보를 갖는 객체를 생성해보세요.

항목	정보
종목명	삼성전자
종목코드	005930
PER	15.79
PBR	1.33
배당수익률	2.83

정답확인

```
삼성 = Stock("삼성전자", "005930", 15.79, 1.33, 2.83)
print(삼성.배당수익률)
```

---

268 객체의 속성 수정

PER, PBR, 배당수익률은 변경될 수 있는 값입니다. 이 값을 변경할 때 사용하는 set_per, set_pbr, set_dividend 메서드를 추가하세요.

정답확인

```
class Stock:
    def __init__(self, name, code, per, pbr, dividend):
        self.name = name
        self.code = code
        self.per = per
        self.pbr = pbr
        self.dividend = dividend

    def set_name(self, name):
        self.name = name

    def set_code(self, code):
        self.code = code

    def get_name(self):
        return self.name

    def get_code(self):
        return self.code

    def set_per(self, per):
        self.per = per

    def set_pbr(self, pbr):
        self.pbr = pbr

    def set_dividend(self, dividend):
        self.dividend = dividend
```

```
삼성 = Stock("삼성전자", "005930", 15.79, 1.33, 2.83)
삼성.set_per(12.75)
print(삼성.per)
```

---

270 여러 종목의 객체 생성

아래의 표를 참조하여 3종목에 대해 객체를 생성하고 이를 파이썬 리스트에 저장하세요. 파이썬 리스트에 저장된 각 종목에 대해 for 루프를 통해 종목코드와 PER을 출력해보세요.

종목명	종목코드	PER	PBR	배당수익률
삼성전자	005930	15.79	1.33	2.83
현대차	005380	8.70	0.35	4.27

LG전자 066570 317.34 0.69 1.37
정답확인
종목 = []

삼성 = Stock("삼성전자", "005930", 15.79, 1.33, 2.83)
현대차 = Stock("현대차", "005380", 8.70, 0.35, 4.27)
LG전자 = Stock("LG전자", "066570", 317.34, 0.69, 1.37)

종목.append(삼성)
종목.append(현대차)
종목.append(LG전자)

```
for i in 종목:
    print(i.code, i.per)        # i-> Stock 클래스의 객체를 바인딩하기 때문
```

---

271 Account 클래스
은행에 가서 계좌를 개설하면 은행이름, 예금주, 계좌번호, 잔액이 설정됩니다. Account 클래스를 생성한 후 생성자를 구현해보세요. 생성자에서는 예금주와 초기 잔액만 입력 받습니다. 은행이름은 SC은행으로 계좌번호는 3자리-2자리-6자리 형태로 랜덤하게 생성됩니다.

은행이름: SC은행
계좌번호: 111-11-111111
정답확인

```
import random

class Account:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance
        self.bank = "SC은행"
        num1 = random.randint(0, 999)
        num2 = random.randint(0, 99)
        num3 = random.randint(0, 999999)

        num1 = str(num1).zfill(3)       # 1 -> '1' -> '001'
        num2 = str(num2).zfill(2)       # 1 -> '1' -> '01'
        num3 = str(num3).zfill(6)       # 1 -> '1' -> '0000001'
        self.account_number = num1 + '-' + num2 + '-' + num3  # 001-01-000001

kim = Account("김민수", 100)
print(kim.name)
```

```
print(kim.balance)
print(kim.bank)
print(kim.account_number)
```

```
김민수
100
SC은행
858-91-548764
```

==========

272 클래스 변수

클래스 변수를 사용해서 Account 클래스로부터 생성된 계좌 객체의 개수를 저장하세요.

정답확인

```
import random

class Account:
    # class variable
    account_count = 0

    def __init__(self, name, balance):
        self.name = name
        self.balance = balance
        self.bank = "SC은행"

        num1 = random.randint(0, 999)
        num2 = random.randint(0, 99)
        num3 = random.randint(0, 999999)

        num1 = str(num1).zfill(3)       # 1 -> '1' -> '001'
        num2 = str(num2).zfill(2)       # 1 -> '1' -> '01'
        num3 = str(num3).zfill(6)       # 1 -> '1' -> '0000001'
        self.account_number = num1 + '-' + num2 + '-' + num3  # 001-01-000001

        Account.account_count += 1


kim = Account("김민수", 100)
print(Account.account_count)
lee = Account("이민수", 100)
print(Account.account_count)
```

1
2

==========
273 클래스 변수 출력
Account 클래스로부터 생성된 계좌의 개수를 출력하는 get_account_num() 메서드를 추가하세요.

정답확인
```
import random

class Account:
    # class variable
    account_count = 0

    def __init__(self, name, balance):
        self.name = name
        self.balance = balance
        self.bank = "SC은행"

        # 3-2-6
        num1 = random.randint(0, 999)
        num2 = random.randint(0, 99)
        num3 = random.randint(0, 999999)

        num1 = str(num1).zfill(3)       # 1 -> '1' -> '001'
        num2 = str(num2).zfill(2)       # 1 -> '1' -> '01'
        num3 = str(num3).zfill(6)       # 1 -> '1' -> '0000001'
        self.account_number = num1 + '-' + num2 + '-' + num3  # 001-01-000001
        Account.account_count +=1

    @classmethod
    def get_account_num(cls):
        print(cls.account_count)      # Account.account_count


kim = Account("김민수", 100)
lee = Account("이민수", 100)
kim.get_account_num()
```

2
==========
274 입금 메서드
Account 클래스에 입금을 위한 deposit 메서드를 추가하세요. 입금은 최소 1원 이상만 가능합니다.

정답확인
```
import random

class Account:
    # class variable
    account_count = 0

    def __init__(self, name, balance):
        self.name = name
        self.balance = balance
        self.bank = "SC은행"

        # 3-2-6
        num1 = random.randint(0, 999)
        num2 = random.randint(0, 99)
        num3 = random.randint(0, 999999)

        num1 = str(num1).zfill(3)       # 1 -> '1' -> '001'
        num2 = str(num2).zfill(2)       # 1 -> '1' -> '01'
        num3 = str(num3).zfill(6)       # 1 -> '1' -> '0000001'
        self.account_number = num1 + '-' + num2 + '-' + num3  # 001-01-000001
        Account.account_count +=1

    @classmethod
    def get_account_num(cls):
        print(cls.account_count)      # Account.account_count

    def deposit(self, amount):
        if amount >= 1:
            self.balance += amount
```
---
275 출금 메서드
Account 클래스에 출금을 위한 withdraw 메서드를 추가하세요. 출금은 계좌의 잔고 이상으로 출금할 수는 없습니다.

정답확인

```
import random


class Account:
    # class variable
    account_count = 0

    def __init__(self, name, balance):
        self.name = name
        self.balance = balance
        self.bank = "SC은행"

        # 3-2-6
        num1 = random.randint(0, 999)
        num2 = random.randint(0, 99)
        num3 = random.randint(0, 999999)

        num1 = str(num1).zfill(3)  # 1 -> '1' -> '001'
        num2 = str(num2).zfill(2)  # 1 -> '1' -> '01'
        num3 = str(num3).zfill(6)  # 1 -> '1' -> '0000001'
        self.account_number = num1 + '-' + num2 + '-' + num3  # 001-01-000001
        Account.account_count += 1

    @classmethod
    def get_account_num(cls):
        print(cls.account_count)  # Account.account_count

    def deposit(self, amount):
        if amount >= 1:
            self.balance += amount

    def withdraw(self, amount):
        if self.balance > amount:
            self.balance -= amount

k = Account("kim", 100)
k.deposit(100)
k.withdraw(90)
print(k.balance)
```

110
==========
276 정보 출력 메서드
Account 인스턴스에 저장된 정보를 출력하는 display_info() 메서드를 추가하세요. 잔고는 세 자리마다 쉼표를 출력하세요.

은행이름: SC은행
예금주: 파이썬
계좌번호: 111-11-111111
잔고: 10,000원
정답확인
import random

class Account:
    # class variable
    account_count = 0

    def __init__(self, name, balance):
        self.name = name
        self.balance = balance
        self.bank = "SC은행"

        # 3-2-6
        num1 = random.randint(0, 999)
        num2 = random.randint(0, 99)
        num3 = random.randint(0, 999999)

        num1 = str(num1).zfill(3) # 1 -> '1' -> '001'
        num2 = str(num2).zfill(2) # 1 -> '1' -> '01'
        num3 = str(num3).zfill(6) # 1 -> '1' -> '0000001'
        self.account_number = num1 + '-' + num2 + '-' + num3 # 001-01-000001
        Account.account_count += 1

    @classmethod
    def get_account_num(cls):
        print(cls.account_count) # Account.account_count

    def deposit(self, amount):
        if amount >= 1:
            self.balance += amount

```
    def withdraw(self, amount):
        if self.balance > amount:
            self.balance -= amount

    def display_info(self):
        print("은행이름: ", self.bank)
        print("예금주: ", self.name)
        print("계좌번호: ", self.account_number)
        print("잔고: ", self.balance)


p = Account("파이썬", 10000)
p.display_info()
```

==========

은행이름: SC은행
예금주: 파이썬
계좌번호: 387-97-896159
잔고: 10000
---

277 이자 지급하기
입금 횟수가 5회가 될 때 잔고를 기준으로 1%의 이자가 잔고에 추가되도록 코드를 변경해보세요.

정답확인

```
import random


class Account:
    # class variable
    account_count = 0

    def __init__(self, name, balance):
        self.deposit_count = 0

        self.name = name
        self.balance = balance
        self.bank = "SC은행"

        # 3-2-6
```

```
        num1 = random.randint(0, 999)
        num2 = random.randint(0, 99)
        num3 = random.randint(0, 999999)

        num1 = str(num1).zfill(3)  # 1 -> '1' -> '001'
        num2 = str(num2).zfill(2)  # 1 -> '1' -> '01'
        num3 = str(num3).zfill(6)  # 1 -> '1' -> '0000001'
        self.account_number = num1 + '-' + num2 + '-' + num3  # 001-01-000001
        Account.account_count += 1

    @classmethod
    def get_account_num(cls):
        print(cls.account_count)  # Account.account_count

    def deposit(self, amount):
        if amount >= 1:
            self.balance += amount

            self.deposit_count += 1
            if self.deposit_count % 5 == 0:          # 5, 10, 15
                # 이자 지금
                self.balance = (self.balance * 1.01)


    def withdraw(self, amount):
        if self.balance > amount:
            self.balance -= amount

    def display_info(self):
        print("은행이름: ", self.bank)
        print("예금주: ", self.name)
        print("계좌번호: ", self.account_number)
        print("잔고: ", self.balance)

p = Account("파이썬", 10000)
p.deposit(10000)
p.deposit(10000)
p.deposit(10000)
p.deposit(5000)
p.deposit(5000)
print(p.balance)
```

```
---------------------------
50500.0
================================
```

278 여러 객체 생성

Account 클래스로부터 3개 이상 인스턴스를 생성하고 생성된 인스턴스를 리스트에 저장해보세요.

정답확인

```
import random


class Account:
    # class variable
    account_count = 0

    def __init__(self, name, balance):
        self.deposit_count = 0

        self.name = name
        self.balance = balance
        self.bank = "SC은행"

        # 3-2-6
        num1 = random.randint(0, 999)
        num2 = random.randint(0, 99)
        num3 = random.randint(0, 999999)

        num1 = str(num1).zfill(3)  # 1 -> '1' -> '001'
        num2 = str(num2).zfill(2)  # 1 -> '1' -> '01'
        num3 = str(num3).zfill(6)  # 1 -> '1' -> '0000001'
        self.account_number = num1 + '-' + num2 + '-' + num3  # 001-01-000001
        Account.account_count += 1

    @classmethod
    def get_account_num(cls):
        print(cls.account_count)  # Account.account_count

    def deposit(self, amount):
        if amount >= 1:
            self.balance += amount
```

```
            self.deposit_count += 1
            if self.deposit_count % 5 == 0:          # 5, 10, 15
                # 이자 지급
                self.balance = (self.balance * 1.01)


    def withdraw(self, amount):
        if self.balance > amount:
            self.balance -= amount

    def display_info(self):
        print("은행이름: ", self.bank)
        print("예금주: ", self.name)
        print("계좌번호: ", self.account_number)
        print("잔고: ", self.balance)

data = []  ?? error
k = Account("KIM", 10000000)
l = Account("LEE", 10000)
p = Account("PARK", 10000)

data.append(k)
data.append(l)
data.append(p)

print(data)
```

---

279 객체 순회

반복문을 통해 리스트에 있는 객체를 순회하면서 잔고가 100만원 이상인 고객의 정보만 출력하세요.

정답확인

```
import random


class Account:
    # class variable
    account_count = 0

    def __init__(self, name, balance):
```

```
        self.deposit_count = 0

        self.name = name
        self.balance = balance
        self.bank = "SC은행"

        # 3-2-6
        num1 = random.randint(0, 999)
        num2 = random.randint(0, 99)
        num3 = random.randint(0, 999999)

        num1 = str(num1).zfill(3)  # 1 -> '1' -> '001'
        num2 = str(num2).zfill(2)  # 1 -> '1' -> '01'
        num3 = str(num3).zfill(6)  # 1 -> '1' -> '0000001'
        self.account_number = num1 + '-' + num2 + '-' + num3  # 001-01-000001
        Account.account_count += 1

    @classmethod
    def get_account_num(cls):
        print(cls.account_count)  # Account.account_count

    def deposit(self, amount):
        if amount >= 1:
            self.balance += amount

            self.deposit_count += 1
            if self.deposit_count % 5 == 0:          # 5, 10, 15
                # 이자 지금
                self.balance = (self.balance * 1.01)


    def withdraw(self, amount):
        if self.balance > amount:
            self.balance -= amount

    def display_info(self):
        print("은행이름: ", self.bank)
        print("예금주: ", self.name)
        print("계좌번호: ", self.account_number)
        print("잔고: ", self.balance)
```

```
data = []
k = Account("KIM", 10000000)
l = Account("LEE", 10000)
p = Account("PARK", 10000)
data.append(k)
data.append(l)
data.append(p)

for c in data:
    if c.balance >= 1000000:
        c.display_info()
====================================
은행이름:  SC은행
예금주:  KIM
계좌번호:  984-27-907021
잔고:  10000000
=================================
```

280 입출금 내역

입금과 출금 내역이 기록되도록 코드를 업데이트 하세요. 입금 내역과 출금 내역을 출력하는 deposit_history와 withdraw_history 메서드를 추가하세요.

정답확인

```
import random


class Account:
    # class variable
    account_count = 0

    def __init__(self, name, balance):
        self.deposit_count = 0
        self.deposit_log = []
        self.withdraw_log = []

        self.name = name
        self.balance = balance
        self.bank = "SC은행"

        # 3-2-6
        num1 = random.randint(0, 999)
        num2 = random.randint(0, 99)
```

```
        num3 = random.randint(0, 999999)

        num1 = str(num1).zfill(3)  # 1 -> '1' -> '001'
        num2 = str(num2).zfill(2)  # 1 -> '1' -> '01'
        num3 = str(num3).zfill(6)  # 1 -> '1' -> '0000001'
        self.account_number = num1 + '-' + num2 + '-' + num3  # 001-01-000001
        Account.account_count += 1

    @classmethod
    def get_account_num(cls):
        print(cls.account_count)  # Account.account_count

    def deposit(self, amount):
        if amount >= 1:
            self.deposit_log.append(amount)
            self.balance += amount

            self.deposit_count += 1
            if self.deposit_count % 5 == 0:          # 5, 10, 15
                # 이자 지급
                self.balance = (self.balance * 1.01)


    def withdraw(self, amount):
        if self.balance > amount:
            self.withdraw_log.append(amount)
            self.balance -= amount

    def display_info(self):
        print("은행이름: ", self.bank)
        print("예금주: ", self.name)
        print("계좌번호: ", self.account_number)
        print("잔고: ", self.balance)

    def withdraw_history(self):
        for amount in self.withdraw_log:
            print(amount)

    def deposit_history(self):
        for amount in self.deposit_log:
            print(amount)
```

```
k = Account("Kim", 1000)
k.deposit(100)
k.deposit(200)
k.deposit(300)
k.deposit_history()

k.withdraw(100)
k.withdraw(200)
k.withdraw_history()
```

```
100
200
300
100
200
```

==========

281 클래스 정의

다음 코드가 동작하도록 차 클래스를 정의하세요.

```
>> car = 차(2, 1000)
>> car.바퀴
2
>> car.가격
1000
```

정답확인

```
class 차:
    def __init__(self, 바퀴, 가격):
        self.바퀴 = 바퀴
        self.가격 = 가격


car = 차(2, 1000)
print(car.바퀴)
print(car.가격)
```

---

282 클래스 상속

차 클래스를 상속받은 자전차 클래스를 정의하세요.

정답확인

```
class 차:
    def __init__(self, 바퀴, 가격):
        self.바퀴 = 바퀴
        self.가격 = 가격


class 자전차(차):
    pass
```

---

283 클래스 상속

다음 코드가 동작하도록 자전차 클래스를 정의하세요. 단 자전차 클래스는 차 클래스를 상속받습니다.

```
>> bicycle = 자전차(2, 100)
>> bicycle.가격
100
```

정답확인

```
class 차:
    def __init__(self, 바퀴, 가격):
        self.바퀴 = 바퀴
        self.가격 = 가격


class 자전차(차):
    def __init__(self, 바퀴, 가격):
        self.바퀴 = 바퀴
        self.가격 = 가격


bicycle = 자전차(2, 100)
print(bicycle.가격)
```

---

284 클래스 상속

다음 코드가 동작하도록 자전차 클래스를 정의하세요. 단 자전차 클래스는 차 클래스를 상속받습니다.

```
>> bicycle = 자전차(2, 100, "시마노")
>> bicycle.구동계
시마노
```

정답확인

```
class 차:
    def __init__(self, 바퀴, 가격):
        self.바퀴 = 바퀴
        self.가격 = 가격


class 자전차(차):
    def __init__(self, 바퀴, 가격, 구동계):
        super().__init__(바퀴, 가격)
        #차.__init__(self, 바퀴, 가격)
        self.구동계 = 구동계


bicycle = 자전차(2, 100, "시마노")
print(bicycle.구동계)
print(bicycle.바퀴)
```

---

4) Child 클래스의 __init__() 메소드내에서 **super(). __init__()**을 입력하면 Parent 클래스의 __init__()에 있는 클래스 변수들을 가지고 올 수 있습니다. 5) 이를 볼때 super()는 상속받은 부모 클래스를 의미한다고 볼 수 있습니다

시마노
2

==========

286 부모 클래스 생성자 호출
다음 코드가 동작하도록 자전차 클래스를 수정하세요.

```
>> bicycle = 자전차(2, 100, "시마노")
>> bicycle.정보()
바퀴수 2
가격 100
```

정답확인

```
class 차:
    def __init__(self, 바퀴, 가격):
        self.바퀴 = 바퀴
        self.가격 = 가격

    def 정보(self):
        print("바퀴수 ", self.바퀴)
        print("가격 ", self.가격)
```

```
class 자동차(차):
    def __init__(self, 바퀴, 가격):
        super().__init__(바퀴, 가격)

class 자전차(차):
    def __init__(self, 바퀴, 가격, 구동계):
        super().__init__(바퀴, 가격)
        self.구동계 = 구동계

bicycle = 자전차(2, 100, "시마노")
bicycle.정보()
```

---

287 부모 클래스 메서드 호출

자전차의 정보() 메서드로 구동계 정보까지 출력하도록 수정해보세요.

```
>> bicycle = 자전차(2, 100, "시마노")
>> bicycle.정보()
바퀴수 2
가격 100
구동계 시마노
```

정답확인

```
class 차:
    def __init__(self, 바퀴, 가격):
        self.바퀴 = 바퀴
        self.가격 = 가격

    def 정보(self):
        print("바퀴수 ", self.바퀴)
        print("가격 ", self.가격)

class 자동차(차):
    def __init__(self, 바퀴, 가격):
        super().__init__(바퀴, 가격)

class 자전차(차):
    def __init__(self, 바퀴, 가격, 구동계):
        super().__init__(바퀴, 가격)
        self.구동계 = 구동계

    def 정보(self):
        super().정보()
```

```
        print("구동계 ", self.구동계)

bicycle = 자전차(2, 100, "시마노")
bicycle.정보()
```

바퀴수 2
가격 100
구동계 시마노

==========

288 메서드 오버라이딩

다음 코드의 실행 결과를 예상해보세요.

```
class 부모:
  def 호출(self):
    print("부모호출")

class 자식(부모):
  def 호출(self):
    print("자식호출")
나 = 자식()
나.호출()
```

정답확인

```
class 부모:
    def 호출(self):
        print("부모호출")

class 자식(부모):
    def 호출(self):
        print("자식호출")


나 = 자식()
나.호출()
```

자식호출

==========

289 생성자

다음 코드의 실행 결과를 예상해보세요.

```
class 부모:
  def __init__(self):
```

```
    print("부모생성")

class 자식(부모):
  def __init__(self):
    print("자식생성")
나 = 자식()
```

정답확인

```
class 부모:
    def __init__(self):
        print("부모생성")

class 자식(부모):
    def __init__(self):
        print("자식생성")

나 = 자식()
```

자식생성

==========

290 부모클래스 생성자 호출

다음 코드의 실행 결과를 예상해보세요.

```
class 부모:
  def __init__(self):
    print("부모생성")

class 자식(부모):
  def __init__(self):
    print("자식생성")
    super().__init__()

나 = 자식()
```

정답확인

```
class 부모:
    def __init__(self):
        print("부모생성")

class 자식(부모):
    def __init__(self):
        print("자식생성")
        super().__init__()
```

나 = 자식()

자식생성
부모생성
==========
291 파일 쓰기
바탕화면에 '매수종목1.txt' 파일을 생성한 후 다음과 같이 종목코드를 파일에 써보세요.

005930
005380
035420
정답확인
f = open("C:/Users/pc/Desktop/매수종목1.txt", mode="wt", encoding="utf-8")
f.write("005930\n")
f.write("005380\n")
f.write("035420")
f.close()

==========
292 파일 쓰기
바탕화면에 '매수종목2.txt' 파일을 생성한 후 다음과 같이 종목코드와 종목명을 파일에 써보세요.

005930 삼성전자

005380 현대차
035420 NAVER
정답확인

```
f = open("C:/Users/pc/Desktop/매수종목2.txt", mode="wt", encoding="utf-8")
f.write("005930 삼성전자\n")
f.write("005380 현대차\n")
f.write("035420 NAVER\n")
f.close()
```

005930 삼성전자
005380 현대차
035420 NAVER

==========

293 CSV 파일 쓰기

바탕화면에 '매수종목.csv' 파일을 생성한 후 다음과 같이 종목코드와 종목명을 파일에 써보세요. 인코딩은 'cp949'를 사용해야합니다.

	A	B	C
1	종목명	종목코드	PER
2	삼성전자	005930	15.79
3	NAVER	035420	55.82

정답확인

```
import csv

f = open("C:/Users/pc/Desktop/매수종목.csv", mode="wt", encoding="cp949", newline='')
writer = csv.writer(f)
writer.writerow(["종목명", "종목코드", "PER"])
writer.writerow(["삼성전자", "005930", 15.59])
writer.writerow(["NAVER", "035420", 55.82])
f.close()
```

매수종목.csv

종목명	종목코드	PER
삼성전자	5930	15.59
NAVER	35420	55.82

---

294 파일 읽기

바탕화면에 생성한 '매수종목1.txt' 파일을 읽은 후 종목코드를 리스트에 저장해보세요.

005930
005380
035420

정답확인

```
f = open("C:/Users/pc/Desktop/매수종목1.txt", encoding="utf-8")
lines = f.readlines()    # python list

codes = []
for line in lines:
    code = line.strip()  #'\n'
    codes.append(code)

print(codes)

f.close()
```

['005930', '005380', '035420']

==========

295 파일 읽기

바탕화면에 생성한 '매수종목2.txt' 파일을 읽은 후 종목코드와 종목명을 딕셔너리로 저장해보세요. 종목명을 key로 종목명을 value로 저장합니다.

005930 삼성전자
005380 현대차
035420 NAVER

정답확인

```
f = open("C:/Users/pc/Desktop/매수종목2.txt", encoding="utf-8")
lines = f.readlines()

data = {}
for line in lines:
    line = line.strip()      # '\n' 제거
    k, v = line.split()
    print(k, v)
```

```
    data[k] = v

print(data)
f.close()
```

005930 삼성전자
005380 현대차
035420 NAVER
{'005930': '삼성전자', '005380': '현대차', '035420': 'NAVER'}
==========
296 예외처리
문자열 PER (Price to Earning Ratio) 값을 실수로 변환할 때 에러가 발생합니다. 예외처리를 통해 에러가 발생하는 PER은 0으로 출력하세요.

```
per = ["10.31", "", "8.00"]

for i in per:
    print(float(i))
```
정답확인
```
per = ["10.31", "", "8.00"]

for i in per:
    try:
        print(float(i))
    except:
        print(0)
```
---
297 예외처리 및 리스트에 저장
문자열로 표현된 PER 값을 실수로 변환한 후 이를 새로운 리스트에 저장해보세요.

```
per = ["10.31", "", "8.00"]

for i in per:
    print(float(per))
```
정답확인
```
per = ["10.31", "", "8.00"]
new_per = []

for i in per:
    try:
        v = float(i)
```

```
    except:
        v = 0
    new_per.append(v)

print(new_per)
```

[10.31, 0, 8.0]
==========

298 특정 예외만 처리하기
어떤 값을 0으로 나누면 ZeroDivisionError 에러가 발생합니다. try ~ except로 모든 에러에 대해 예외처리하지 말고 ZeroDivisionError 에러만 예외처리해보세요.

정답확인
```
try:
    b = 3 / 0
except ZeroDivisionError:
    print("0으로 나누면 안되요")
```

0으로 나누면 안되요
==========
299 예외의 메시지 출력하기
다음과 같은 코드 구조를 사용하면 예외 발생 시 에러 메시지를 변수로 바인딩할 수 있습니다.

```
try:
    실행코드
except 예외 as 변수:
    예외처리코드
```
리스트의 인덱싱에 대해 에러를 출력해보세요.

```
data = [1, 2, 3]

for i in range(5)
    print(data[i])
```
정답확인
```
data = [1, 2, 3]

for i in range(5):
    try:
        print(data[i])
```

```
    except IndexError as e:
        print(e)
```

1
2
3
list index out of range
list index out of range
==========
300 try, except, else, finally 구조 사용해보기
파이썬 예외처리는 다음과 같은 구조를 가질 수 있습니다.

```
try:
    실행 코드
except:
    예외가 발생했을 때 수행할 코드
else:
    예외가 발생하지 않았을 때 수행할 코드
finally:
    예외 발생 여부와 상관없이 항상 수행할 코드
```

아래의 코드에 대해서 예외처리를 사용하고 try, except, else, finally에 적당한 코드를 작성해봅시다. else와 finally는 적당한 문구를 print하시면 됩니다.

```
per = ["10.31", "", "8.00"]

for i in per:
    print(float(per))
```

정답확인

```
per = ["10.31", "", "8.00"]

for i in per:
    try:
        print(float(i))
    except:
        print(0)
    else:
        print("clean data")
    finally:
        print("변환 완료")
```
==========
10.31

clean data
변환 완료
0
변환 완료
8.0
clean data
변환 완료
==========